

SurfSLAM: Sim-to-Real Underwater Stereo Reconstruction For Real-Time SLAM

Onur Bagoren* Seth Isaacson* Sacchin Sundar Yung-Ching Sun Anja Sheppard
 Haoyu Ma Abrar Shariff Ram Vasudevan Katherine A. Skinner

<https://umfieldrobotics.github.io/SurfSLAM/>

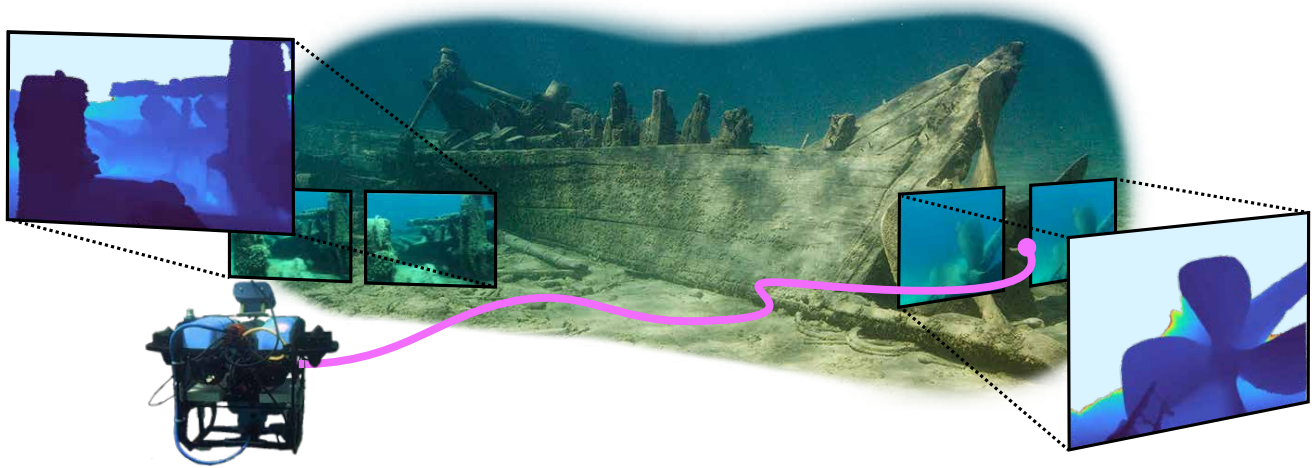


Figure 1. SurfSLAM performs real-time SLAM in challenging underwater environments by fusing proprioceptive data with global registration. Accurate global registration is enabled by an underwater stereo disparity estimation algorithm that is fine-tuned via a novel sim-to-real pipeline. Illustrated is our BlueROV data collection platform navigating a shipwreck. Two stereo frames along the robot’s path (drawn in pink) are visualized. Background image courtesy of the National Oceanic and Atmospheric Administration Thunder Bay National Marine Sanctuary.

Abstract

Localization and mapping are core perceptual capabilities for underwater robots. Stereo cameras provide a low-cost means of directly estimating metric depth to support these tasks. However, despite recent advances in stereo depth estimation on land, computing depth from image pairs in underwater scenes remains challenging. In underwater environments, images are degraded by light attenuation, visual artifacts, and dynamic lighting conditions. Further-

more, real-world underwater scenes frequently lack rich texture useful for stereo depth estimation and 3D reconstruction. As a result, stereo estimation networks trained on in-air data cannot transfer directly to the underwater domain. In addition, there is a lack of real-world underwater stereo datasets for supervised training of neural networks. Poor underwater depth estimation is compounded in stereo-based Simultaneous Localization and Mapping (SLAM) algorithms, making it a fundamental challenge for underwater robot perception. To address these challenges, we propose a novel framework that enables sim-to-real training of underwater stereo disparity estimation networks using simulated data and self-supervised finetuning. We leverage our learned depth predictions to develop SurfSLAM, a novel framework for real-time underwater SLAM that fuses stereo cameras with IMU, barometric, and Doppler Velocity

*Denotes equal contribution

This work was supported by the National Science Foundation under Award No. 2337774.

All authors are with the Department of Robotics, University of Michigan, Ann Arbor, MI 48109, USA

Corresponding author e-mail: obagoren@umich.edu

Log (DVL) measurements. Lastly, we collect a challenging real-world dataset of shipwreck surveys using an underwater robot. Our dataset features over 24,000 stereo pairs, along with high-quality, dense photogrammetry models and reference trajectories for evaluation. Through extensive experiments, we demonstrate the advantages of the proposed training approach on real-world data for improving stereo estimation in the underwater domain and for enabling accurate trajectory estimation and 3D reconstruction of complex shipwreck sites. Code, real-world underwater data, and simulated training data will be made publicly available.

1. Introduction

Underwater simultaneous localization and mapping (SLAM) is a key challenge in marine robotics, with wide-ranging applications such as underwater inspection [23] and semantic mapping [63]. Vision plays a crucial role in this context. Cameras are relatively low-cost sensors, and stereo cameras can provide metric depth information useful for reconstructing the scene geometry. However, underwater imaging presents several fundamental differences from in-air imagery. The presence of haze, attenuation, dynamic lighting, and suspended particles reduces contrast and detail. Further, large areas of real-world underwater scenes often lack distinctive, trackable textures useful for estimating geometry [3, 34]. These underwater effects can vary significantly between locations and conditions, making it challenging to develop perception systems that remain robust across diverse underwater environments [27].

Classical, hand-designed methods for stereo depth estimation struggle in textureless and hazy regions; as a result, they do not generalize well to the underwater domain [21]. For the in-air domain, significant attention has been devoted to the monocular depth estimation problem [84]. In the underwater domain, several works have adapted in-air monocular depth estimation networks to account for underwater image characteristics [86–88]. Still, monocular depth estimation cannot reliably provide metric-scale depth. Recent deep stereo networks achieve excellent performance in estimating in-air metric depth [28, 77, 79]. However, as we demonstrate in this work, their accuracy degrades significantly underwater due to the domain shift induced by haze, attenuation, and dynamic lighting. Several works have proposed learning-based solutions to underwater depth estimation [65, 78, 85], but a lack of foundation-scale underwater training data and limitations in simulation fidelity have limited their accuracy.

In this work, we propose **SurfSLAM**, illustrated in Figure 1. SurfSLAM is a novel framework for **Sim-to-real Underwater stereo Reconstruction For real-time SLAM**. Our sim-to-real stereo depth estimation pipeline leverages knowledge of underwater imaging effects to recover dense,

reliable metric depth from underwater stereo imagery. We demonstrate that our sim-to-real training enables state-of-the-art stereo depth estimation in the underwater domain. As a result, the dense depth maps estimated by our approach provide strong geometric priors to improve navigation and mapping capabilities. Our proposed SLAM framework fuses depth predictions with measurements from an IMU, Doppler Velocity Log (DVL), and barometer to track vehicle pose. The vehicle pose and dense depth maps are then used to estimate a dense 3D map of the scene.

Our method is evaluated on a dataset of long-duration shipwreck surveys for visual-acoustic-inertial SLAM. Our dataset includes reference 3D reconstructions and vehicle trajectories to support qualitative and quantitative evaluation. We present extensive experiments demonstrating that SurfSLAM improves both trajectory accuracy and reconstruction quality for underwater SLAM in challenging real-world environments compared to existing state-of-the-art methods. Code, model weights, real-world underwater data, and simulated training data will be made publicly available upon publication.

2. Related Work

SurfSLAM is a method that combines underwater SLAM, stereo depth estimation, and the transfer of deep learning methods to the underwater domain. We review the relevant literature below.

2.1. Visual SLAM

The task of estimating camera motion from stereo images, optionally fused with inertial sensing, has received enormous attention. ORB-SLAM and its variants [8] operate by matching and triangulating image features, tracking camera motion relative to feature maps with inertial priors, and jointly optimizing trajectories with a map via bundle adjustment [8]. These methods also implement a keypoint-based visual place recognition method that helps mitigate long-term drift. While ORB-SLAM3 provides outstanding trajectory accuracy in visually rich scenes, it is prone to tracking loss in featureless regions and does not output a dense map useful for scene understanding. As an alternative to feature-based methods, Droid-SLAM uses a learned optical flow module on images for dense correspondence matching and a learned pose update operator for incrementally estimating the state and producing a dense map representation [68]. Still, Droid-SLAM struggles in feature-sparse regions and lacks loop closure detection. More recent approaches have applied large vision foundation models as the backbone to visual SLAM. Prominent work leverages vision transformers (ViTs) pretrained on large-scale simulated in-air data for zero-shot feature matching or 3D reconstruction [33, 74], with MAST3R-SLAM integrating MAST3R pointmaps into a monocular keyframe-based esti-

mator [40]. VGGT-SLAM instead jointly estimates structure and motion on the SL(4) group from uncalibrated sequences [37]. These systems remain image-centric, heavily rely on ViT, and perform poorly in underwater domains where images fall out of the training distribution. In contrast, our proposed method, SurfSLAM, fuses acoustic-inertial data with learned stereo depth estimation and visual place recognition to produce accurate trajectories in both featureless and feature-rich environments.

2.2. Underwater Visual SLAM

Early underwater visual SLAM focused on monocular-based navigation for large-scale inspections using pose-graph formulations [15, 31, 32, 48]. These works introduced informative visual constraint selection based on local geometry and demonstrated robust state estimation by fusing vision with DVL and inertial measurements for large-scale inspections [5, 43]. Later works studied how visual characteristics in underwater settings, such as directional light, could be leveraged within the visual front end, improving both tracking and mapping performance [48].

Later works tightly coupled acoustic and inertial sensors with visual information for more robust state estimation and mapping performance [4, 72, 80]. Pan et al. present a tightly integrated imaging sonar-IMU-camera system [44], while SVIn2 fuses a barometer and a mechanically scanning sonar into a sonar-visual-acoustic state estimator [49]. Thoms et al. integrate a camera, DVL, IMU, and LiDAR for an autonomous surface vehicle, providing foundational work on the integration of the DVL into a pose-graph-based formulation [69]. AQUA-SLAM similarly integrates a DVL into a visual-inertial odometry (VIO) framework for underwater applications, with online extrinsic calibration [82]. These approaches yield robust pose estimates but produce sparse maps that lack the dense information available from stereo cameras present in each of the methods.

Song et al. produce dense maps through depth fusion of stereo depth maps along trajectories estimated by an acoustic-inertial tracker [66]. Wang et al. perform a similar process by using a Sum of Absolute Differences to compute stereo disparities, combined with poses estimated from SVIn2 [76].

Our work incorporates metric depth measurements estimated by a stereo network to form global registration factors, and combines them into an acoustic-visuo-inertial pose graph. As a result, SurfSLAM both maintains accurate tracking in featureless regions via acoustic-inertial fusion and mitigates long-term drift via global place recognition. Furthermore, with dense stereo depth estimation, SurfSLAM is able to output a dense map of complex underwater environments.

2.3. Stereo Depth Estimation

Classical stereo and 3D reconstruction methods rely on hand-crafted pipelines for matching cost computation, disparity optimization, and refinement [6, 22, 50, 56]. Commonly used methods include EpicFlow-based matching [6], [73], and Agisoft Metashape [2]. These approaches tend to be highly sensitive to hyperparameters, which may need to be re-tuned depending on the scene. Other methods focus on matching keypoints between images, then triangulating and optimizing 3D map points [58, 59]. While this may provide good geometric accuracy, these methods cannot operate in textureless regions. Furthermore, they can take several hours to several days to run on real-world scenes, limiting their utility in the field.

The introduction of deep learning for stereo depth estimation offers an alternative to hand-crafted approaches. RAFT-Stereo [35] employs recurrent feature matching and iterative refinement with a typical encoder-decoder network architecture to produce disparity maps from a stereo image pair. However, this approach often requires finetuning on a target domain for optimal performance. The recent release of FoundationStereo [77] has demonstrated the potential of models trained on large-scale simulated datasets to achieve excellent cross-domain, generalizable zero-shot performance on in-air data. Foundation stereo leverages a pre-trained and frozen foundation model encoder from DepthAnything V2 [83]. The foundation model features are fused with side-tuned CNN features and processed by a combination of convolutions, transformer mechanisms, and iterative refinement to produce high-quality disparity estimates [77]. DEFOM-Stereo [28] also uses a foundation model to generate monocular depth estimation cues, but leverages a RAFT-Stereo backbone. IGEV++ [79] forgoes foundation model encoders and instead introduces multi-range geometry encoding volumes to capture different levels of encoding granularity, leading to strong performance in large-disparity scenes. While IGEV++ cannot achieve the level of detail and accuracy of [28, 77], it produces competitive results with significantly lower memory footprint and inference time than foundation model-based methods. A prominent feature of deep learning methods for stereo prediction networks is the iterative refinement process, typically modeled as a variant of a gated recurrent unit (GRU) layer [77] or an incremental disparity update step [28]. The iterative refinement step updates an initial disparity estimate based on correlation features extracted early in the network architecture, providing context relevant for refining disparity estimates. Still, since these methods are developed and trained for in-air stereo depth estimation, they lack generalizability to underwater conditions such as turbidity, low light, caustics, and attenuation. The lack of large-scale training sets for training underwater stereo networks limits the ability to re-train these networks on real stereo imagery

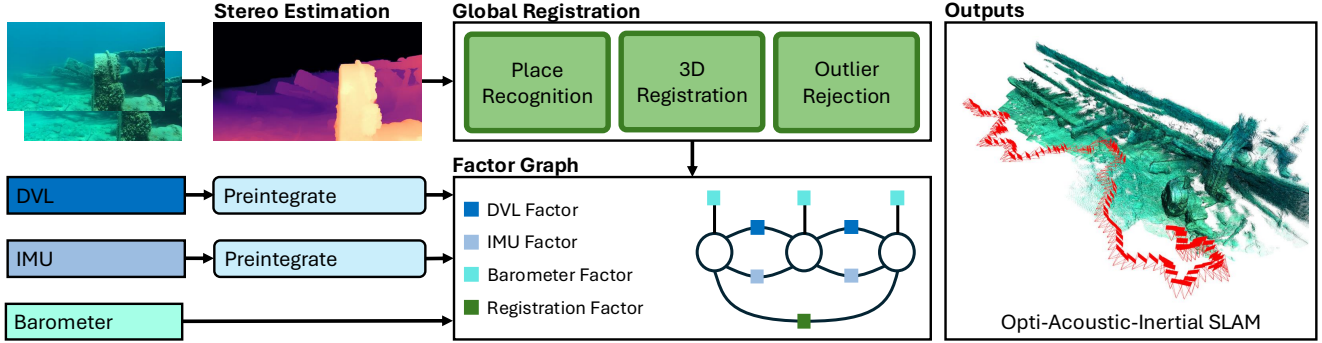


Figure 2. An overview of SurfSLAM. We take as input measurements from a barometer, IMU, DVL, and a stereo image pair. We maintain an acoustic-inertial pose graph that preintegrates measurements to maintain a pose throughout operation. In parallel, we use our finetuned underwater stereo network to produce metric depth maps. These depth estimates and stereo images are used to perform geometric tracking to perform global registration, and reduce drift over operation, producing accurate trajectories and dense maps during operation.

from underwater scenes.

2.4. Deep Learning for Underwater Stereo

A class of work that aims to overcome data scarcity in underwater stereo matching trains a network with direct supervision, using augmentations to make the training data more closely resemble the appearance of underwater images [36, 85, 89]. In [89], the authors employ a style transfer network to transform a reference underwater image into a target in-air stereo pair image with ground-truth disparity, and then train the network using these augmented images. Other works have leveraged a physics-based approach, following underwater image formation models to augment large in-air datasets with water column effects [36].

While simulated data mitigate the data scarcity problem, accurately simulating the complex underwater image formation remains challenging. Self-supervised approaches aim to overcome this by leveraging stereo image pair consistency objectives. UW StereoNet [65] pre-trains an unsupervised Siamese network for disparity estimation on in-air data and then fine-tunes it on underwater data using a self-supervised objective of warping the stereo pairs onto each other and computing a photometric error between them. More recently, StereoAdapter [78] targets underwater domain shift by parameter-efficiently adapting a monocular foundation encoder and a self-supervised objective that similarly minimizes a photometric error between stereo pairs with additional losses to handle occlusions and improve depths at edges. In [78], a dataset of 40,000 simulated images is also released, consisting of augmentations, texture maps and environments to better match the underwater image conditions, and perform pre-training with this dataset.

We propose a fusion of the two approaches. We first develop a novel, comprehensive train-time augmentation pipeline to generate realistic underwater images from large-scale in-air datasets, producing simulated data to train a

model with direct supervision on the stereo disparity estimates. This train-time augmentation is made possible due to the simulated data being coupled with the stereo intrinsic and extrinsic parameters, a key addition coupled with our simulated data. As a fine-tuning approach, similar to [65, 78], we employ a warping loss for self-supervised training. As a novel contribution over existing methods, we propose additional regularization to guide the model to make correct predictions when the background (i.e., water column) is present in the stereo image pairs. The depths from the finetuned model are then used in our SurfSLAM framework to enable tracking and mapping in environments where the geometric cues allow for place recognition, global registration, and loop closures.

3. Method Overview

We propose **SurfSLAM**, a novel framework for **Sim-to-real Underwater stereo Reconstruction For real-time SLAM**. An overview of the proposed method is shown in Figure 2. First, our sim-to-real stereo depth estimation pipeline leverages knowledge of underwater imaging effects to recover dense, reliable metric depth from underwater stereo imagery. Then, SurfSLAM fuses our dense stereo depth predictions using global registration with measurements from an IMU, DVL, and barometer for tracking vehicle pose. The poses and depth maps are then used to build a dense 3D reconstruction of the scene.

The discussion of the proposed technical approach is organized as follows: Subsection 3.1 provides the notation and coordinate conventions used throughout the paper. Section 4 presents our proposed methodology for sim-to-real training of underwater stereo disparity estimation. Then, Section 5 describes our SurfSLAM system, which fuses stereo depth estimation with acoustic and inertial sensing for real-time localization and dense mapping of complex underwater scenes.

3.1. Preliminaries

3.1.1. Notation

We denote scalars as lowercase italics ($x, y, z, \dots$), matrices as uppercase bold ($\mathbf{X}, \mathbf{Y}, \mathbf{Z}, \dots$) and vectors as lowercase bold ($\mathbf{x}, \mathbf{y}, \mathbf{z}, \dots$). Sensor measurements are denoted as capitalized calligraphic ($\mathcal{X}, \mathcal{Y}, \mathcal{Z}, \dots$), and estimated variables are denoted with a bar $\bar{x}$.

3.1.2. Frame Definitions

The reference frames are shown in Figure 3, where red indicates the x -axis, green the y -axis, and blue the z -axis. The reference frames include the fixed world frame, $\mathbf{W}$, and the North-East-Down (NED) frame, $\mathbf{N}$. The base frame, $\mathbf{B}$, and IMU frame, $\mathbf{I}$, are shown to be coincident, similar to that of [16], and the sensor frames are represented as the DVL frame, $\mathbf{D}$, the barometer frame, $\mathbf{P}$, and the left camera frame, $\mathbf{C}$. All transformations and their notations follow the definitions in [17].

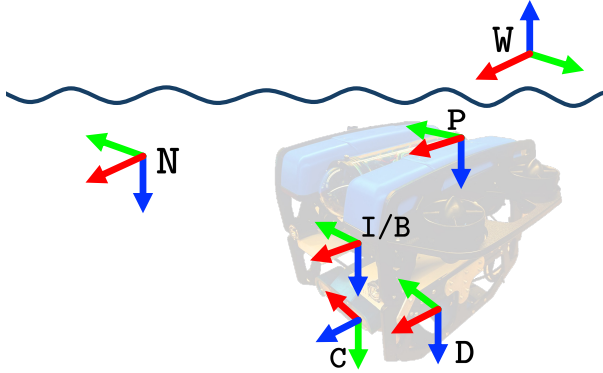


Figure 3. Coordinate frames of the robot. Red indicates the x -axis, green the y -axis, and blue the z -axis. The IMU ($\mathbf{I}$) and base ($\mathbf{B}$) frames are coincident. The DVL ($\mathbf{D}$) and barometer ($\mathbf{P}$) frames share the same frame convention and the camera frame ($\mathbf{C}$) follows the OpenCV frame convention [1].

4. Underwater Stereo Estimation

This section introduces our proposed method for learning-based underwater stereo depth estimation, including data generation, data augmentations, implementation details, and training loss.

4.1. Simulated Data Generation

Motivated by the lack of real-world underwater stereo datasets with ground truth depth, and inspired by the success of stereo disparity estimation in [77], we leverage simulated data for network training. To generate training data, we first leverage the Omniverse RTX renderer [41] to generate rendered images in an IsaacSim environment [42], as shown in the first column of Figure 4. Objects and

scene assets are collected from Epic Fab [14] and Sketchfab [64], totaling eight scenes. These scenes are selected for their similarity in terrain properties (i.e., rocky and sandy seafloors) and structure (i.e., open water, caves). The assets are selected to reflect common underwater structures, including large and small shipwrecks, rocks spanning a range of scales and textures, and marine vegetation. During the initial simulated data generation stage, the position of the stereo camera pair is randomly sampled from a pre-recorded trajectory in each scene, and a random orientation at the position is applied to ensure diversity of views of the scene. With each render, the stereo pair image is generated, along with the camera intrinsics, depth image, and scene normals.

We note that when rendering the initial simulated images, we do not apply any underwater effects. While there has been a recent rise in GPU-accelerated simulators for the underwater domain [47, 53, 67], simulated effects are limited to haze and attenuation. We also seek to model caustics, dynamic lighting, and suspended particles. Thus, we propose an augmentation pipeline that takes in simulated in-air images and applies train-time augmentations. This has the added benefit of randomizing augmentations at training time, yielding multiple configurations of underwater image parameters for each simulated image.

4.2. Underwater Data Augmentation

This section presents a training-time augmentation pipeline that transforms the simulated in-air stereo imagery into physically plausible underwater renderings while preserving dense disparity ground truth. We explicitly model dominant components of underwater image formation and apply them as composable operators to the simulated dataset. Concretely, the pipeline comprises four augmentation modules: (1) ambient water-column illumination, (2) surface-induced caustics from refracted sunlight, (3) directional and diffuse sunlight, and (4) specular highlights from suspended sediment particles. The cumulative effect of these operators on the input images is illustrated in Figure 4. Note that since the presented augmentations only require the simulated in-air stereo images, ground-truth depths, the camera intrinsics, and the stereo extrinsics, it is broadly transferable to other in-air datasets that contain dense ground-truth depth.

4.2.1. Water Column Effects

We model water-column effects using the light transport model described in [57, 61]. For an in-air image, J_c , with depth map, $\mathbf{z}$, the simulated underwater pixel intensity, I_c , at location $\mathbf{x} = (u, v)$ is given by

$$I_c(\mathbf{x}, \mathbf{z}) = J_c(\mathbf{x}) \cdot t_c(\mathbf{z}) + B_{c,\infty} \cdot (1 - t_c(\mathbf{z})) \quad (1)$$

$$t_c(\mathbf{z}) = \exp(-\beta_c \mathbf{z}) \quad (2)$$

where β_c is a per-channel attenuation coefficient for $c \in \{R, G, B\}$ and $B_{c,\infty}$ is the per-channel veiling-light

TABLE 1

A summary of underwater SLAM and stereo datasets. We propose two new datasets: a large simulated dataset with comprehensive underwater augmentation and a real-world dataset. ✓* denotes that the dataset will be released upon publication. Photogrammetry is shortened to PGM and simulation is shortened to Sim. for the ground truth column.

	Name	Camera	Splits	No. Images	Publicly Avail.	Ground Truth	Water Effects			Additional Sensors		
							Turbid	Low Light	Dir. Light	DVL	IMU	Baro.
Stereo	UWStereoNet [65]	Stereo	Train	4,047	✓	LiDAR	✓					
			Test	15	✓							
	SQUID [6]	Stereo	-	57	✓	PGM	✓					
	FLSea [50]	Stereo	-	19,596	✓	PGM	✓		✓			
	SOTRUE [38]	Stereo	-	8,497	✓	Encoder	✓		✓			
	UWStereo [36]	Sim.	-	29,568	✗	Sim.	✓	✓				
	StereoAdapter [78]	Sim.	Both	40,000	✓	Sim.	✓	✓	✓			
	UWSim (Ours)	Sim.	-	105,600	✓*	Sim.	✓	✓	✓			
SLAM			Train	19,950	✓*	-	✓	✓	✓			
			Test	1,486		PGM	✓	✓	✓			
	UWslam [7]	Stereo	-	3,111	✓	PGM	✓					
	FLSea [50]	Mono.	-	22,456	✓	PGM	✓		✓		✓	
	SVIn2 [49]	Custom	-	25,145	✓	PGM	✓	✓	✓		✓	✓
	Tank [81]	Stereo	-	40,649	✓	AprilTag	✓	✓	✓	✓	✓	✓
	SUDS (Ours)	Stereo	-	29,319	✓*	PGM	✓	✓	✓	✓	✓	✓

(backscatter) term at infinite range. We randomly sample β_c according to Jerlov water types [27], and we sample $B_{c,\infty}$ over a range of conditions to span diverse underwater appearances at each time we apply the augmentation. This approach is flexible enough to sample and generate the appearance of additional water column parameters for targeted augmentations to emulate various water conditions extending beyond the selected Jerlov water types [62]. The effect of this model applied to a simulated in-air image is shown in the second column of Figure 4.

4.2.2. Caustics

Caustics arise from sunlight refraction at the dynamic water surface [70]; while analytic, surface-based caustic rendering is available in real-time graphics engines [67], it is impractical to integrate such pipelines within a train-time augmentation. We instead model underwater caustics via texture mapping. Given an image J_c with depth map $\mathbf{z}(\mathbf{x})$, we compute per pixel scene normals, $\mathbf{N}(\mathbf{x})$, and apply a caustic texture map, T_{caustic} , under a planar light source aligned with the scene $+z$ -axis with direction $\mathbf{L}_z$. The resulting caustic intensity image is

$$I_{\text{caustic}}(\mathbf{x}) = T_{\text{caustic}}(\mathbf{x}) \max(0, \langle \mathbf{L}_z, \mathbf{N}(\mathbf{x}) \rangle). \quad (3)$$

This single-channel image is used to relight the image J_c . For stereo pairs, we use the known stereo extrinsics to warp the texture map for consistent caustic projection. The effect of the caustic simulation is shown in the third column in Figure 4. In practice, the normals are computed using

Kaolin [26].

4.2.3. Directional Light

Inspired by the presence of directional lighting in underwater datasets (see Table 1) and by methods that explicitly exploit lighting for reconstruction [54], we incorporate directional light simulation into our dataset generation.

Given the depth map, $\mathbf{z}(\mathbf{x})$, and its computed normals, $\mathbf{N}(\mathbf{x})$, we place a point light source at position p in the camera frame with pose P and color C_c . For a pixel at location $\mathbf{x} = (u, v)$, the light direction is

$$\mathbf{L}(\mathbf{x}) = p - \mathbf{z}(\mathbf{x}) K^{-1} \begin{bmatrix} u & v & 1 \end{bmatrix}^T. \quad (4)$$

where K is the camera intrinsic matrix. The contribution of the directional light to the original image is

$$I_{\text{light},c}(\mathbf{x}) = \frac{PC_c}{\|\mathbf{L}(\mathbf{x})\|^2} \max(0, \langle \mathbf{L}(\mathbf{x}), \mathbf{N} \rangle) t_c(\|\mathbf{L}(\mathbf{x})\|), \quad (5)$$

where t_c is depth-dependent, per-channel attenuation defined in (2). For a stereo pair, we keep p fixed in the left camera frame and simulate the light contribution on the right image. The effect of directional lighting is shown in the fourth column in Figure 4.

4.2.4. Sunlight

In shallow-water applications, sunlight significantly affects the captured image. This is evident in the real-world image

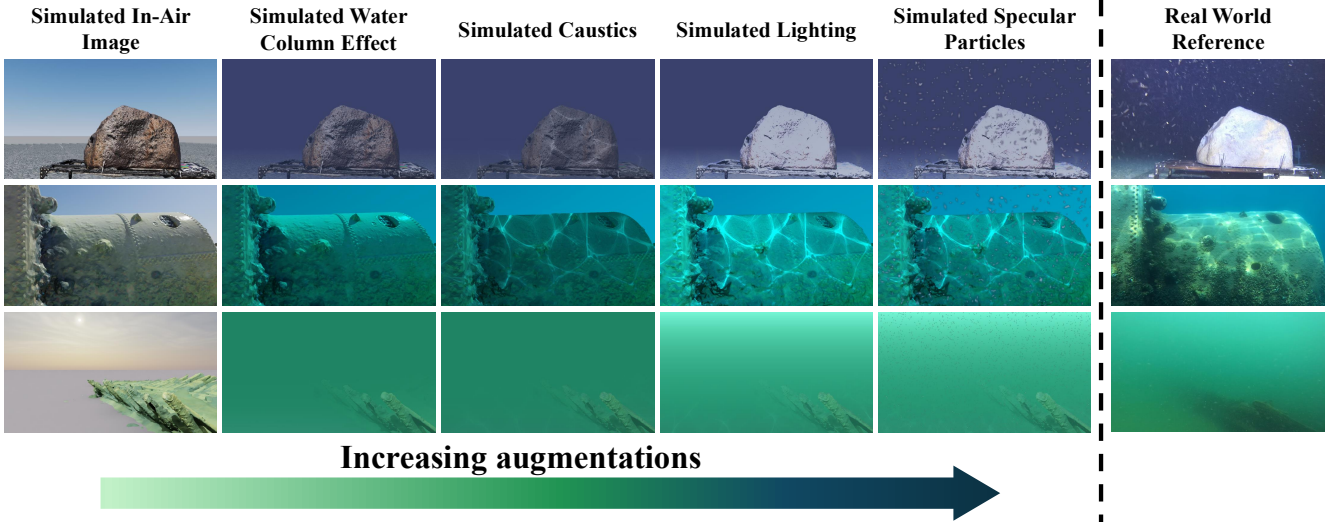


Figure 4. A demonstration of our underwater augmentation pipeline. From left to right, the first five columns demonstrate increasing levels of augmentation. The rightmost column shows the reference image that the augmentations aim to match.

shown in the bottom row of Figure 4. Effects like crepuscular rays are prevalent in these environments, and the strong sunlight source contributes to a gradient of color change in the water column [45]. We model two dominant sunlight modes purely in image space, without relying on the 3D scene geometry: planar and radial illumination.

For planar illumination, we impose a vertical intensity gradient in the image, with pixels near the top receiving the strongest contribution. This is parametrized by the sun color, C_{sun} , the additional brightness, b_{planar} , and saturation distance in image space, d_{sat} . Color and saturation changes are computed consistently with the water column parameters, β_c and B_∞ , used to add the water column effects. For radial illumination, we define C_{sun} and a sun position, p_{sun} , in image coordinates; radial distance from p_{sun} controls the color and intensity change, again tied to β_c and B_∞ . Sampling p_{sun} above the image plane increases the curvature of the halo, mimicking a sun just outside the camera’s field of view. The effect of a planar sunlight on the augmented image is shown in the fourth column in Figure 4.

4.2.5. Particles

Suspended particles are a common feature of underwater imagery, arising from high turbidity or sediment in the water column. We model only particles located between the scene and the camera. To simulate particle appearance, we apply a procedurally generated particle texture map directly in image space. For each particle texture map, we randomly place a set of ellipsoids across the image, varying in orientation and size. Each ellipsoid’s opacity decreases radially outward from its center, simulating the motion blur experienced during image capture. The resulting particle

texture map is additively blended with the base image. The effect of simulated particles is shown in the fifth row of Figure 4.

4.2.6. Full Augmentation Pipeline

For the full augmentation pipeline, shown graphically in Figure 4, we apply all lighting effects first, then add water haze to simulate an underwater image. We define a relit image $I_{\text{relit},c}$ as

$$I_{\text{relit},c}(\mathbf{x}) = J_c(\mathbf{x}) (1 + I_{\text{light},c}(\mathbf{x})) (1 + I_{\text{caustic}}(\mathbf{x})). \quad (6)$$

We apply the water-column model to $I_{\text{relit},c}$ to ensure all illumination is subject to medium effects:

$$I_{\text{water},c}(\mathbf{x}, \mathbf{z}) = I_{\text{relit},c} t_c(\mathbf{z}) + B_\infty (1 - t_c(\mathbf{z})). \quad (7)$$

As a final step, we apply the sunlight and particles directly onto I_{water} . The parameters of each augmentation are randomly sampled to form a large distribution of water effects.

4.3. Stereo Training

In this section, we describe a unified training strategy for fine-tuning existing stereo depth networks in underwater environments. We combine supervised objectives on our augmented synthetic data with self-supervised objectives on real underwater stereo data and apply this strategy across a range of model sizes and architectures [28, 77].

4.3.1. Loss Functions

Stereo networks produce a set of refined disparity estimates $\hat{\mathbf{d}} = (\hat{d}_0, \dots, \hat{d}_R)$, where $\hat{d}_r$ is the final prediction after R

refinement steps. The training loss switches between supervised ($\mathcal{L}_{\text{sup}}$) and self-supervised ($\mathcal{L}_{\text{self}}$) losses depending on whether the ground-truth disparity, d^* , is available. That is,

$$\mathcal{L}(\hat{\mathbf{d}}) = \begin{cases} \mathcal{L}_{\text{sup}}(\hat{\mathbf{d}}, d^*) & d^* \text{ known} \\ \lambda_{\text{self}} \cdot \mathcal{L}_{\text{self}}(\hat{\mathbf{d}}_R) & \text{otherwise} \end{cases} \quad (8)$$

where λ_{self} is a scalar hyperparameter.

4.3.2. Direct Disparity Supervision

Ground-truth disparity, d^* , is available only for simulated data. For training on simulated data, to avoid supervising on regions of the image severely degraded by water column effects, we use the known per-channel attenuation $t_c(\mathbf{x}) = \exp(-\beta_c \mathbf{x})$ from our augmentation pipeline. We mask out regions where $t_c(\mathbf{x}) < t_{\min, c}$ for any channel, retaining only pixels that carry information about the scene geometry.

$$d^*(\mathbf{x}) = d^*(\mathbf{x}) \mathbb{1}[t_c(\mathbf{x}) \geq t_{\min, c}] \quad (9)$$

With this masked ground-truth, we use the same loss function as in [77]:

$$\mathcal{L}_{\text{sup}}(\hat{\mathbf{d}}, d^*) = |\hat{d}_0 - d^*|_{\text{smooth}} + \sum_{r=1}^R \gamma^{R-r} \|\hat{d}_r - d^*\|_1 \quad (10)$$

where $|\cdot|_{\text{smooth}}$ denotes the Smooth L1 loss and $\|\cdot\|_1$ denotes the standard L1 Loss, as described by [18, 77].

4.3.3. Self-Supervised Warping Loss

When ground-truth disparity is not available, we use an alternate supervision strategy. Following several past works [65, 71], we use a self-supervising loss that uses the estimated disparity to warp each image to the frame of the other. That is, given a rectified pair of images (I_L, I_R) along with the rectified intrinsics, K , and the stereo baseline, b , each image is back-projected to 3D by converting the disparities to depths. Then, the 3D points are projected into the frame of the other image and interpolated to form the final image. We denote this operation $\hat{I}_L = \text{warp}(I_R, d)$ and $\hat{I}_R = \text{warp}(I_L, d)$, where the dependence on calibrations is omitted for brevity. In practice, this warp operation is implemented by the open-source Kornia library [52].

Using the estimated images and ground-truth, the following loss is computed:

$$\hat{I}_L = \text{warp}(I_R, \hat{d}_K) \quad \hat{I}_R = \text{warp}(I_L, \hat{d}_K) \quad (11)$$

$$\mathcal{L}_{\text{warp}}(\hat{d}_K) = \frac{1}{2} \left[\mathcal{L}_{\text{DTD}}(\hat{I}_L, I_L) + \mathcal{L}_{\text{DTD}}(\hat{I}_R, I_R) \right], \quad (12)$$

where

$$\mathcal{L}_{\text{DTD}}(I, \hat{I}) = \lambda_{\text{DTD}} \cdot |I - \hat{I}|_1 + (1 - \lambda_{\text{DTD}}) \cdot \text{SSIM}(I, \hat{I}) \quad (13)$$

is the loss function proposed by [71]. Unlike the directly supervised loss in (10), the self-supervised loss is applied only to the final upsampled prediction $\hat{d}_K$.

While [71] applies warping only from the right image to the left, we follow [65] and apply the loss in both directions. Although (12) is written as the average of the two losses, in practice, the warped image may contain invalid pixels at the edges (caused by parts of the scene being out of view in one camera). In such cases, we keep only the valid camera's loss and weight it twice, while discarding the invalid camera's loss.

4.3.4. Occam Regularizer

Even though (12) provides effective self-supervision in textured regions, low-texture areas cause many disparities to produce the same warped image, especially in background regions where disparity should be zero. To resolve this, we add an *Occam regularizer* that favors zero disparity when it yields results similar to larger disparities.

Let $\mathcal{L}_{\text{warp}}(\hat{d}_K)$ be the photometric loss in (12) for the predicted disparity map $\hat{d}_K$. We compute an alternative loss $\mathcal{L}_{\text{warp}}(0)$ corresponding to a *zero-disparity hypothesis* by directly comparing the unwrapped stereo pair. Then, we define

$$\Delta_{\text{occam}}(\hat{d}_K) = \mathcal{L}_{\text{warp}}(\hat{d}_K) - \mathcal{L}_{\text{warp}}(0), \quad (14)$$

which measures the gain from the predicted disparity over zero disparity.

If the prediction is significantly worse than zero disparity, we add a penalty:

$$\mathcal{L}_{\text{occam}}(\hat{d}_K) = \text{ReLU}(\tau_{\text{occam}} + \Delta_{\text{occam}}(\hat{d}_K)), \quad (15)$$

where τ_{occam} is a margin parameter.

4.3.5. Total Loss Function

Finally, as in [71], we add the edge-aware smoothness regularizer, $\mathcal{L}_{\text{smooth}}$, from [19]. Thus, the final self-supervised loss is

$$\begin{aligned} \mathcal{L}_{\text{self}}(\hat{d}_K) = & \lambda_{\text{warp}} \mathcal{L}_{\text{warp}}(\hat{d}_K) \\ & + \lambda_{\text{occam}} \mathcal{L}_{\text{occam}}(\hat{d}_K) \\ & + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}}(\hat{d}_K) \end{aligned} \quad (16)$$

where λ_{warp} , λ_{occam} and λ_{smooth} are scalar weights determined experimentally. In practice, we train the model with two stages. In the first stage, we train using only simulated data for a fixed number of iterations. We then fine-tune on the second stage using a mixture of simulated and real-world data.

5. SurfSLAM

Our proposed algorithm SurfSLAM applies the finetuned stereo depth estimation module and tightly integrates the IMU, DVL, barometer, and stereo depth maps to estimate the robot's pose and to build a dense 3D reconstruction of the scene. We base our tracking system on TURTLMAP [66] due to its metrically consistent odometry in low-texture underwater environments. We treat the stereo-derived geometry as a high-level measurement used to correct drift with global registration factors while relying on acoustic-inertial sensing for locally consistent motion. For the sake of clarity, we describe the full acoustic-inertial-stereo method below.

5.1. State Definition

The robot state at a keyframe i is represented as a combination of the pose, velocity, and sensor biases:

$$\mathbf{x}_i = [\mathbf{R}_i, \mathbf{p}_i, \mathbf{v}_i, \mathbf{b}_i^g, \mathbf{b}_i^a, \mathbf{b}_i^v] \in \text{SO}(3) \times \mathbb{R}^{15}, \quad (17)$$

where $\mathbf{R}_i \in \text{SO}(3)$ and $\mathbf{p}_i \in \mathbb{R}^3$ represent the orientation and position in the NED frame, $\mathbf{v}_i \in \mathbb{R}^3$ represents the body frame linear velocity, $\mathbf{b}_i^g, \mathbf{b}_i^a \in \mathbb{R}^3$ are the IMU gyroscope and accelerometer biases, and $\mathbf{b}_i^v \in \mathbb{R}^3$ is the DVL velocity bias. We maintain the states over a set of keyframes

$$\mathcal{X}_n = \{\mathbf{x}_i\}_{i \in K_n}, \quad (18)$$

where K_n represents the keyframes up until time t_n . In practice, we add keyframes at 1 Hz to balance factor graph complexity, stereo inference time, and estimation accuracy.

5.2. Measurement Definitions

We consider sensor measurements from an IMU, a DVL, and a barometer between two consecutive keyframes i and j . For a stereo camera, we consider the measurements between a query keyframe at time q and a matched keyframe at time m .

The depth estimates from the stereo camera are denoted $\mathcal{S}_{ij}$. The IMU measurements are denoted as $\mathcal{I}_{ij}$, and are composed of a set of angular velocity $\tilde{\omega} \in \mathbb{R}^3$ and linear acceleration $\tilde{\mathbf{a}} \in \mathbb{R}^3$ measurements. The DVL measurements, denoted $\mathcal{D}_{ij}$, consist of a linear velocity $\tilde{\mathbf{v}} \in \mathbb{R}^3$ measurement, and the barometer measurement, denoted as $\mathcal{P}_{ij}$ is composed of a pressure measurement, $\tilde{z}$. The full set of measurements is denoted

$$\mathcal{Z}_n = \{\mathcal{S}_{qm}, \mathcal{I}_{ij}, \mathcal{D}_{ij}, \mathcal{P}_{ij}\}_{i,j,q,m \in K_n}. \quad (19)$$

5.3. Acoustic-Inertial Pose Graph Formulation

We follow the formulation presented in [66] for acoustic-inertial odometry. We denote the subset of acoustic-inertial

measurements as $\underline{\mathcal{Z}}_k$ consisting of the IMU, DVL and barometer until time t_k :

$$\underline{\mathcal{Z}}_k = \{\mathcal{I}_{ij}, \mathcal{D}_{ij}, \mathcal{P}_{ij}\}_{i,j \in K_k}. \quad (20)$$

The estimator optimizes state estimates to maximize the posterior distribution given the measurements $\underline{\mathcal{Z}}_k$ and a prior distribution over the states $\mathcal{X}_0$:

$$\mathcal{X}_k^* = \underset{\mathcal{X}_k}{\operatorname{argmax}} p(\mathcal{X}_k | \underline{\mathcal{Z}}_k) \quad (21)$$

$$\propto \underset{\mathcal{X}_k}{\operatorname{argmax}} p(\mathcal{X}_0) p(\underline{\mathcal{Z}}_k | \mathcal{X}_k) \quad (22)$$

We assume that the sensor measurements are conditionally independent and have zero-mean Gaussian noise. Following this, we can transform the objective function into a weighted least squares problem by minimizing the log-likelihood of each measurement and the prior over the states, weighted by their associated uncertainties. This objective is given as

$$\begin{aligned} \mathcal{X}_k^* = \underset{\mathcal{X}_k}{\operatorname{argmin}} & \|\mathbf{r}_0\|_{\Sigma_0}^2 \\ & + \sum_{i,j \in K_k} \left(\|\mathbf{r}_{\mathcal{I}_{ij}}\|_{\Sigma_{\mathcal{I}_{ij}}}^2 + \|\mathbf{r}_{\mathcal{D}_{ij}}\|_{\Sigma_{\mathcal{D}_{ij}}}^2 + \|\mathbf{r}_{\mathcal{P}_{ij}}\|_{\Sigma_{\mathcal{P}_{ij}}}^2 \right), \end{aligned} \quad (23)$$

where $\|\cdot\|_{\Sigma}^2$ denotes the Mahalanobis distance with covariance Σ , and $\mathbf{r}$ denotes the residual associated with the factor for the measurements $\underline{\mathcal{Z}}_k$ and the prior belief $\mathcal{X}_0$. In practice, this optimizer is solved iteratively as measurements arrive using iSAM2 [30]. The residuals and noise models for the sensors are derived in [16, 66, 69].

5.4. Global Registration

This section describes how the tracking method from [66] is augmented with global registration. Global registration is performed in three steps: (1) global place recognition, (2) visual feature matching, and (3) geometric alignment. Each step is described below.

5.4.1. Global Place Recognition

Let K_q denote the set of keyframes up to a time $t_q \leq t_k$, and let $k_q \in K_q$ be the q^{th} keyframe that is used to query the place recognition system. The left image of k_q , denoted $I_{q,l}$, is first histogram equalized using Contrast-Limited Adaptive Histogram Equalization (CLAHE) [46]. Then, SuperPoint features (as proposed and trained by [11, 55]) are extracted from the equalized image. The local feature descriptors from the SuperPoint features are accumulated into an image-level descriptor using the Vector of Locally Aggregated Descriptors (VLAD) method [29]. In practice, the K-nearest neighbors process within VLAD is implemented

via [13]. The image-level descriptors computed by VLAD are compared with other keyframes that are at least 20 seconds old. The top three most similar images are retrieved and passed to the visual feature matching step.

5.4.2. Visual Feature Matching

We denote a matched image candidate as $I_{m,l}$. Initial feature matching is performed using SuperGlue [55]. Outliers are rejected by computing the essential matrix between the query and match images using RANSAC. If RANSAC estimates sufficient inliers, the match candidate proceeds to the geometric validation step.

5.4.3. Geometric Validation

Using the stereo estimation described in 4, a depth image is computed for both the query and matched images. Using these depth images, the filtered 2D correspondences estimated by SuperGlue are lifted to 3D. These 3D feature correspondences are used to estimate a 3D transformation, $\mathbf{T}_{qm}^C$, between the two keyframes. This is again performed using RANSAC to reject poor correspondences. If insufficient inliers are detected, the registration is rejected. Otherwise, the RANSAC-estimated transform is refined using GICP [60] on the full point clouds estimated by the stereo depth estimation network, yielding a final geometric relative pose measurement $\tilde{\mathbf{T}}_{qm}^C \in \text{SE}(3)$, which we simply denote as the stereo measurement, $\mathcal{S}_{qm}$.

While global place recognition produces three candidate frames, at most one is passed to the optimizer. That is, the first match candidate to survive the above heuristics is passed to the pose graph, while any remaining candidates are discarded.

5.4.4. Pose Graph Formulation

We incorporate $\mathcal{S}_{qm}$ as a factor added to the acoustic-inertial tracking to constrain the relative motion between the keyframes. Before adding the registration to the pose graph, a final probabilistic test is performed. A stereo factor, $\mathcal{S}_{qm}$, is added to the graph after keyframes at time t_q and t_m have been optimized using the acoustic-inertial factors described in Section 5.2. As a result, the tracking system can be queried for a belief over T_{qm}^C , including mean $\mu_{T_{qm}^C}$ and marginal covariance $\Sigma_{T_{qm}^C}$. We compute the Mahalanobis distance between the stereo registration result and the tracker’s estimate according to

$$d_{qm} = \left\| T_{qm}^C - \mu_{T_{qm}^C} \right\|_{\Sigma_{T_{qm}^C}}^2 \quad (24)$$

Registrations with $d_{qm} > 2.5$ are treated as outliers and discarded; only those satisfying $d_{qm} \leq 2.5$ are added to the pose graph as stereo factors $\mathcal{S}_{qm}$.

In practice, each of the outlier rejection methods is tuned to add as many registrations to the factor graph as possible. To deal with the noisy estimates resulting from this

strategy, registration factors are weighted by a robust Huber kernel [24] in the backend to reject incorrect registrations that survive the outlier rejection process.

The final objective becomes

$$\begin{aligned} \mathcal{X}_k^* = \underset{\mathcal{X}_k}{\operatorname{argmin}} & \left\| \mathbf{r}_0 \right\|_{\Sigma_0}^2 \\ & + \sum_{i,j \in \mathbf{K}_k} \left(\left\| \mathbf{r}_{\mathcal{I}_{ij}} \right\|_{\Sigma_{\mathcal{I}_{ij}}}^2 + \left\| \mathbf{r}_{\mathcal{D}_{ij}} \right\|_{\Sigma_{\mathcal{D}_{ij}}}^2 + \left\| \mathbf{r}_{\mathcal{P}_{ij}} \right\|_{\Sigma_{\mathcal{P}_{ij}}}^2 \right) \end{aligned} \quad (25)$$

$$+ \sum_{q,m \in \mathbf{K}_q} \rho \left(\left\| \mathbf{r}_{\mathcal{S}_{qm}} \right\|_{\Sigma_{\mathcal{S}_{qm}}}^2 \right), \quad (26)$$

where the stereo pose registration residual $\mathbf{r}_{\mathcal{S}_{qm}}$ is defined as

$$\mathbf{r}_{\mathcal{S}_{qm}} = \operatorname{Log} \left(\tilde{T}_{qm}^{-1} \tilde{T}_{qm} \right) \in \mathfrak{se}(3) \quad (27)$$

where $\tilde{T}_{qm}$ represents the pose measurement from the GICP estimate, $\tilde{T}_{qm}$ represents the estimated relative pose, Log represents the mapping from the Lie group $\text{SE}(3)$ to its Lie algebra $\mathfrak{se}(3)$ and ρ denotes the robust Huber kernel.

5.5. Mapping

For mapping, we adopt a similar approach to that of [68] and maintain the stereo depth at each keyframe as a map frame primitive. Each incoming keyframe contributes a depth image that is back-projected into a local point cloud using the calibrated stereo intrinsics and extrinsics. These keyframe point sets are accumulated to form an explicit geometric reconstruction of the environment.

To address noise introduced to the pointcloud by depth discontinuities, we apply a lightweight, GPU-based outlier rejection and noise cleaning step prior to the map frame construction. Specifically, we apply an edge cleaning, followed by a statistical outlier removal and radius outlier removal step. This results in a map with minimal noise and errors and maintains the global registrations that anchor the pose during the tracking phase of SurfSLAM.

6. Experiments

6.1. Data Collection and Generation

Real-world underwater stereo data was collected at the Thunder Bay National Marine Sanctuary in Alpena, MI, with multiple shipwreck sites and in an outdoor test tank across multiple days. The data collection platform is a human-operated BlueROV2 with a forward-facing stereo camera. Field sequences span depths from 4 to 20 m and rely on natural illumination; additional sequences were acquired around an artificial rock target in an outdoor pool (both day and night), where the BlueROV2 lights provide additional lighting as needed. The dataset exhibits a broad

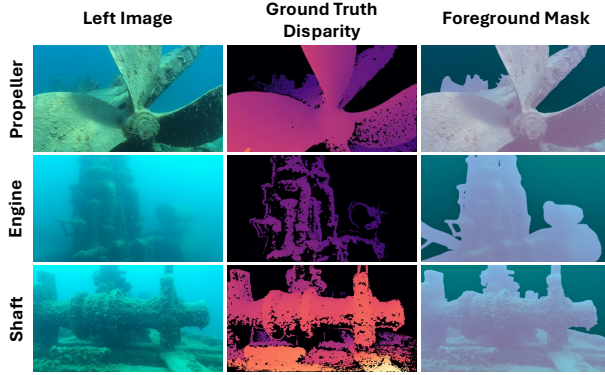


Figure 5. An example from three of the nine stereo sequences is shown. The first column shows the left rectified image; the second column shows the disparity estimated from our metric photogrammetry pipeline; the last column shows the manually-annotated foreground mask overlaid on the left image. The disparity color maps are so that lighter (yellow) is higher disparity and darker (purple) is lower disparity, with black being zero disparity.

range of turbidity, caustics, lighting, and suspended particulates (see the rightmost column of Figure 4 and leftmost column of Figure 5).

Our collected dataset, which we call **Stereo Underwater Dataset for Shipwrecks (SUDS)**, contains three subsets: (i) stereo evaluation, (ii) stereo training, and (iii) SLAM evaluation. All three dataset splits were collected at different shipwreck sites. Test data was collected at shipwreck sites that were not seen during training.

Our simulation dataset, which we call **UnderWater Simulation (UWSim)**, contains simulated data of realistic underwater scenes generated as described in Section 4.1 and is used for training our stereo networks. This differs from alternative datasets used in underwater stereo training, which use either simulated images from non-underwater images or lack the visual properties of underwater sequences.

SLAM evaluation comprises three sequences on a single shipwreck under varying haze, texture, and geometry. The dataset statistics are provided in Table 1.

6.1.1. Real-world Stereo Dataset

The real-world stereo training dataset consists of twenty-two real-world sequences without accompanying ground-truth data, intended for self-supervision, with detailed statistics in Table 1. This dataset is composed of a variety of underwater lighting conditions, turbidity levels, and distractors. In addition to the stereo images, this dataset also consists of the stereo intrinsic and extrinsic parameters, making the dataset suitable for self-supervised training, as described in Section 4.3.3.

The SUDS stereo evaluation dataset comprises nine sequences across five shipwreck sites, totaling 1,486 stereo

pairs with labeled ground-truth disparity, poses, and 3D reconstruction. The ground truth for the stereo evaluation dataset was obtained through a metric photogrammetry pipeline. We provide stereo intrinsic and extrinsic parameters and per-frame depth maps through the metric photogrammetry pipeline. Reconstructions of four sequences are shown in the first row of Figure 6.

We note that a key evaluation metric is to assess the success of stereo depth estimation methods in the water column. Stereo depth estimation methods trained in air tend to mistake the water column for scene geometry. To assess the ability of stereo networks to reject the background, we manually annotated each image with foreground-background masks. These manual annotations were augmented with automated ground plane annotations. To reconstruct the ground plane, the distance to the ground plane was computed for each camera pose using the DVL’s altitude measurements from its four beams. A plane was fit to these ground measurements and projected into each camera view. The foreground-background masks are used only for evaluation and reporting metrics, and are never used in training.

Sample outputs from the evaluation dataset are shown in Figure 5. All stereo evaluations are conducted on our real-world dataset.

6.1.2. Real-world SLAM Datasets

With SUDS, we provide three sequences for evaluating SurfSLAM. Each sequence includes 1080p stereo video at 30 fps with time-synchronized IMU, DVL, and barometer measurements. The IMU is recorded at 200 Hz, the DVL at 8 Hz and the barometer at 5 Hz.

Ground truth is generated by taking a downsampled 5 fps version of the 1080p stereo video and running the metric photogrammetry pipeline, detailed in Section 6.2.1. This ensured that even in regions where there is low texture, the photogrammetry would track the camera poses and yield a near-full trajectory of the robot with minimal dropouts. To the extent of our knowledge, the SLAM sequence of the SUDS dataset presents the first publicly released in-the-wild dataset with ground truth trajectory and maps for a robot equipped with an IMU, DVL, barometer, and stereo camera. We refer to these three scenes as *Boiler*, *Engine*, and *Long*. A reconstruction of the *Boiler* and *Long* scenes, along with the ground truth camera trajectories, is shown in the second row of Figure 6.

The *Boiler* sequence is a 180-second-long trajectory circling a large boiler followed by a short traversal over it. The *Engine* sequence is a 284-second-long run alongside the engine section of a shipwreck, transitioning from motion along the engine to lateral passes in front of the stern while inspecting the propeller. Of the three sequences, the *Engine* sequence is the most consistently visually rich sequence. The *Long* sequence is a 550-second-long survey spanning long distances along the shipwreck, including ex-

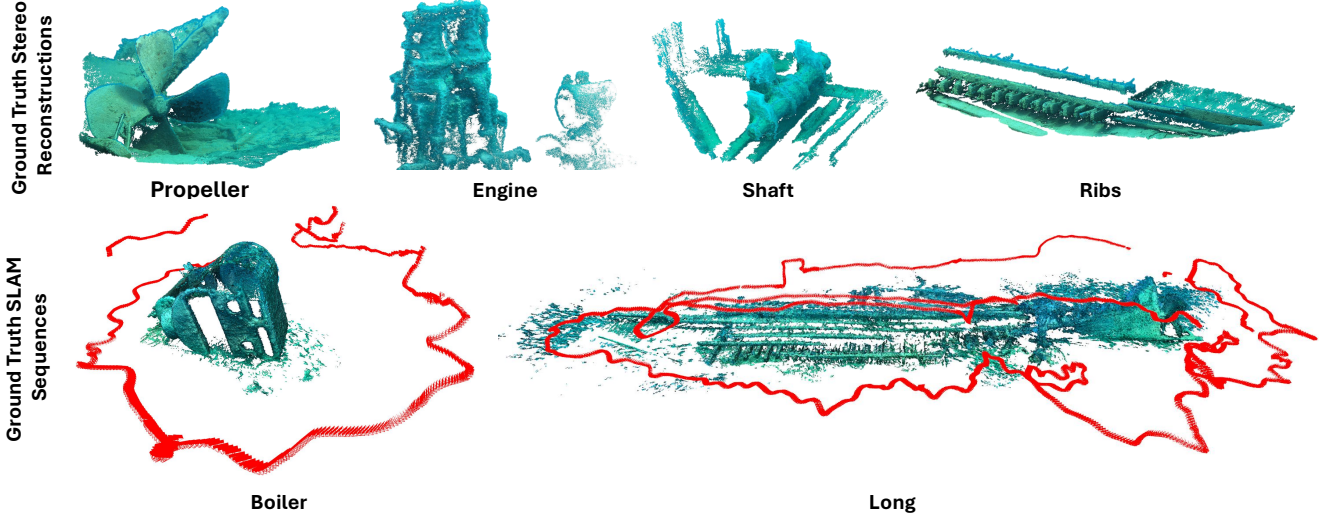


Figure 6. A visualization of a subset of the ground truth metric reconstructions from the stereo (first row) and SLAM sequences (second row) used for evaluations. The SLAM sequences show the ground-truth poses as red camera frustums and a sparse COLMAP reconstruction. The Engine SLAM sequence, not pictured, is a second sequence around the right-hand side of the Long reconstruction.

tended intervals over low-texture regions observing the terrain.

6.2. Generating Ground Truth for Real Data

For data collected in the field, ground truth is generated using a metric photogrammetry pipeline.

6.2.1. Metric Photogrammetry Pipeline

We follow a two-step approach to obtain accurate metric reconstructions, camera poses, intrinsic calibration, and extrinsic calibration of both the stereo pair and IMU. First, structure-from-motion (SfM) is run on sequences of uncalibrated stereo images using COLMAP [58, 59]. An approximate intrinsic calibration produced by Kalibr [51] is used as an initial guess for COLMAP. We then apply a custom optimizer to resolve the metric scale of the scene. The formulation of this metric optimization differs between sequences used for SLAM and those used for stereo evaluation.

6.2.2. Stereo-IMU SLAM Metric Optimization

To recover the metric scale of the scene, we develop a custom optimizer. Kalibr [51] is first used to estimate initial camera intrinsics and stereo-IMU extrinsics. Feature tracks are then exported from COLMAP and imported into a factor graph optimizer. IMU data is recovered from data logs and temporally associated with the corresponding images. Following an implementation closely based on [9], map points are marginalized while camera poses, intrinsics, and extrinsics are jointly optimized. This optimization is solved in batch over all SLAM sequences to enforce a consistent calibration.

After batch optimization, the intrinsic and extrinsic calibrations are fixed, and camera poses and map points are refined on a per-sequence basis. During per-sequence refinement, map points are explicitly treated as optimization variables to allow robust kernels to downweight poor correspondences from COLMAP.

6.2.3. Stereo-Only Stereo Metric Optimization

For stereo-only sequences, no IMU data is available due to equipment failure during field testing. As a result, the scene scale cannot be directly optimized when the stereo baseline is unknown. In this case, the stereo baseline is fixed to the value estimated by Kalibr. Without IMU factors, a two-stage refinement is unnecessary. Instead, camera poses, intrinsics, stereo extrinsics, and map points are jointly optimized to minimize reprojection error. This optimization is performed in batch across all stereo sequences.

6.3. Stereo Training Details

We provide details on the training done for fine-tuning existing stereo networks for underwater stereo depth prediction.

6.3.1. Training Datasets

Datasets For Supervised Training Supervised pretraining uses three simulated stereo datasets: UWSim (ours), TartanAir, and FlyingThings3D [39, 75]. We apply the full underwater augmentation pipeline only to UWSim and TartanAir, and keep FlyingThings3D unaugmented to preserve a reference domain. This reduces the effective domain shift introduced by augmentations and helps prevent optimization instabilities that can otherwise lead to divergence and degraded accuracy in model performance.

TABLE 2

Training parameters for the model finetuned for underwater stereo disparity estimation.

Parameter	Description	Value
Learning Rate	Optimizer step size	1e-5
λ_{occam}	Occam loss weight	1.0
τ_{occam}	Occam margin	0.01
λ_{smooth}	Smoothness weight	0.005
λ_{warp}	Photometric loss weight	10.0
$t_{\text{min},c}$	Visibility threshold	0.05

Datasets for Self-Supervised Training For self-supervised training, we utilize real-world SUDS sequences in conjunction with additional in-the-wild underwater stereo data from SVIn2 and UWslam (Lizard Island), leveraging the provided calibrated stereo intrinsic and extrinsic parameters [7, 49]. These sequences include broad viewpoint variation and diverse water and scene conditions, expanding the coverage of underwater appearance distribution used during the self-supervised finetuning.

6.3.2. Model Architecture

We finetune three different models in our stereo training: FoundationStereo, DEFOM with VIT-L weights, and DEFOM with VIT-S weights. We elect to finetune architectures with a transformer backbone, as it has been widely shown that with large-scale datasets with heavy augmentation, vision transformer models (ViTs) scale better compared to alternative architectures [10, 12]. The selected models include both ViT-L and ViT-S weights, with the ViT-L weights targeted towards more accurate, offline tasks and the ViT-S weights targeted for real-time stereo depth estimation, which we show can be used for real-time SLAM. The parameters used to finetune each of the three models are shown in Table 2.

6.4. Baselines

We evaluate the stereo disparity estimation and SLAM accuracy separately against different sets of baselines.

6.4.1. Stereo Baselines

We evaluate the proposed method against state-of-the-art stereo depth estimation networks. These include FoundationStereo [77], DEFOM-Stereo [28], IGEV++ Stereo [79] and RAFT-Stereo [35]. Additionally, we compare our performance with a stereo depth estimation method developed for the underwater domain, trained on data augmented with water column effects only [85]. For models with multiple weights, we report the best observed performance.

6.4.2. SLAM Baselines

For our SLAM evaluation, we compare against existing state-of-the-art methods in VIO and SLAM, includ-

ing SVIn2 [49], a method developed for underwater state estimation, and ORB-SLAM3 [8]. For SVIn2, we compare the maps produced by the method coupled with [76]. We additionally compare our tracking and mapping against Droid-SLAM [68], MAST3R-SLAM [40], and VGGT-SLAM [37], three representative methods for learning-based visual SLAM. Finally, SLAM results are compared against TURTLMAP [66] as a baseline acoustic-inertial fusion method.

6.5. Evaluation Metrics

6.5.1. Disparity Metrics

For disparity evaluation, we use the metrics from FoundationStereo [77]. We report the end-point error (EPE), defined as the average per-pixel disparity error, and BP-X, the percentage of pixels whose predicted disparity deviates from the ground truth by more than X pixels. In addition, we report the D1 metric, which measures the percentage of pixels with disparity errors exceeding both 3 pixels and 5% of the ground-truth disparity.

Stereo metrics are decomposed into three regions using the manually annotated foreground/background masks. *Combined* considers all pixels, *On Geometry* considers pixels with valid ground-truth disparity from the photogrammetry pipeline, and *Water Column* considers the manually labeled water-column pixels where the correct disparity is zero. In all configurations, pixels labeled as foreground that lack a corresponding ground-truth depth are considered holes in the ground truth. As a result, these pixels are excluded from all metrics.

6.5.2. Tracking Metrics

The absolute pose error (APE) is measured between the reference and estimated trajectories. We use the popular Evo package [20] to compute the tracking metrics. Since MAST3R-SLAM and VGGT-SLAM are monocular methods, we compute trajectory metrics for these methods up to scale.

6.5.3. Mapping Metrics

We follow the procedure from [25] to evaluate maps. Each map is first downsampled to a 5cm resolution. Then, the accuracy, completion, precision, and recall are computed. Accuracy measures the mean distance from each point in the estimate to each point in the ground truth: a high accuracy value indicates a large number of points that are far from the ground truth. Completion is the complementary measure that evaluates the mean distance from each ground truth point to each point in the estimate: a high completion value indicates missing portions of the reconstruction. Precision computes the proportion of points in the estimate that are within 0.1m of a point in the ground truth. Finally, recall computes the proportion of points in the ground truth where there is a point in the estimate within 0.1m.

TABLE 3

We separate stereo evaluations into Large Models with ViT-L backbones and Small Models that use ViT-S or no vision transformer. In both cases, ‘Ours’ refers to DEFOM with our simulation and self-supervised training. EPE (End Point Error) is measured in pixels. BP-1.0 and D1 are described in Section 6.5.1. Lower is better for all metrics. The Params columns give the total number of parameters in each model and, separately, the number that are trainable, which discounts any frozen backbones.

	Model	Params (M)		Combined			On Geometry Only			Water Column Only		
	Method	Total	Train.	EPE	BP-1.0	D1	EPE	BP-1.0	D1	EPE	BP-1.0	D1
Large Models	FoundationStereo (ViT-L)[77]	374.52	39.20	30.79	81.82	<u>51.92</u>	<u>1.76</u>	61.75	3.14	55.62	<u>99.68</u>	<u>98.98</u>
	DEFOM (ViT-L) [28]	382.62	47.30	<u>10.61</u>	<u>78.71</u>	52.30	1.74	<u>55.57</u>	<u>3.29</u>	<u>18.83</u>	99.83	99.64
	Ours (ViT-L)	382.62	47.30	1.51	31.32	4.24	1.84	55.05	3.88	1.41	6.71	5.66
Small Models	IGEV++ [79]	14.53	–	23.22	81.65	52.42	<u>1.90</u>	62.69	<u>4.14</u>	43.63	99.75	99.24
	FoundationStereo (ViT-S)[77]	62.34	37.55	29.98	84.64	52.59	1.97	68.39	4.83	56.42	99.67	98.93
	UnderwaterStereo [85]	2.97	–	25.17	81.81	55.51	7.73	62.61	8.75	41.31	<u>99.59</u>	98.67
	DEFOM (ViT-S)[28]	43.29	18.51	<u>4.03</u>	<u>76.10</u>	<u>49.05</u>	2.31	49.06	5.52	<u>6.15</u>	99.84	<u>91.44</u>
	Ours (ViT-S)	43.29	18.51	1.68	36.39	5.37	1.79	<u>59.07</u>	3.46	2.05	17.07	9.58

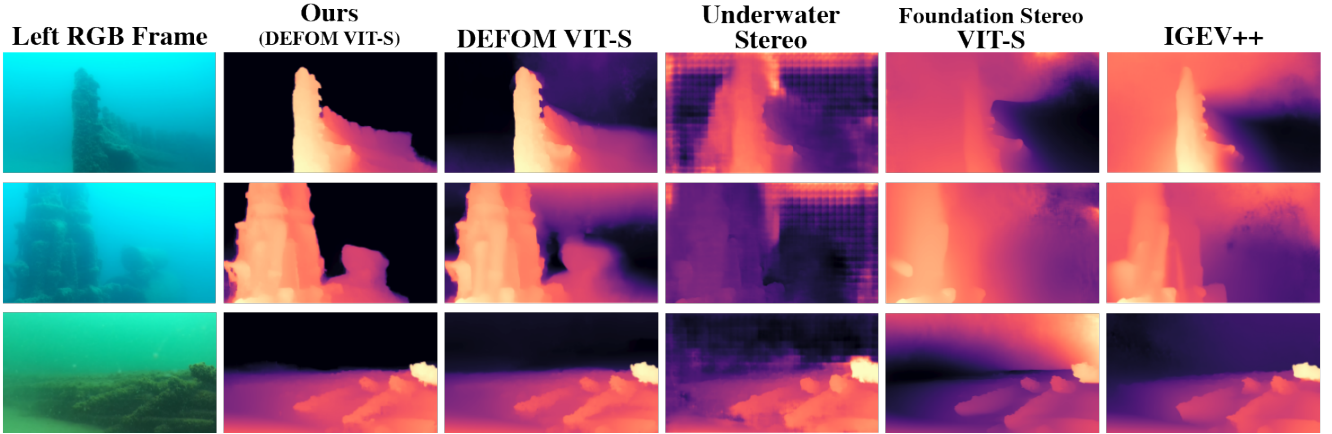


Figure 7. Qualitative comparison of the small models, including those based on the Small Vision Transformer (ViT-S) or without any transformer backbone. Ours is a fine-tuned DEFOM ViT-S, whereas Underwater Stereo and IGEV++ are both convolutional architectures. Of the baseline methods, Underwater Stereo is the only one to be trained on (simulated) underwater images.

6.6. Computation Details

Different compute systems were used for stereo training, stereo inference, metric ground truth reconstructions, and SLAM experiments.

The compute- and memory-intensive ground-truth optimizations were performed on a server equipped with dual AMD Epyc 7742 64-core CPUs and 4 TB of RAM.

The stereo networks were fine-tuned on a system with an AMD Threadripper Pro 7975WX 32-core CPU and four NVIDIA Blackwell Pro 6000 GPUs. Inference was performed on a separate machine with a single NVIDIA Blackwell Pro 6000.

All SLAM baselines were run on a system with an AMD Ryzen 5950X CPU and NVIDIA RTX A6000 GPU. Our SLAM method was evaluated both on that desktop machine and on an NVIDIA Jetson Thor platform, which is suitable

for deployment in the field. The Jetson Thor experiments were conducted on a benchtop using a stream of sensor data, and is intended to evaluate the computational feasibility of running the method on hardware suitable for onboard robotic deployment.

7. Stereo Evaluation Results

Table 3 provides a quantitative comparison of our trained models against baseline methods for stereo depth estimation, where we report the best-performing models from the three that we fine-tuned. We compare our models against three different model classifications: models with a ViT-L backbone [28, 77], ViT-S backbone [28, 77], and models that lack a ViT component [79, 85]. Note that our method uses a DEFOM-Stereo architecture (large and small models) with our proposed train-time augmentations and loss

TABLE 4

We ablate four features: H, haze and water-column simulation; FA, full augmentation (lighting, caustics, particles); WL, inclusion of a self-supervised warping loss; and IA, in-air data to limit drift from the initialization. Performance is measured by EPE (pixels; lower is better). All results are obtained by fine-tuning DEFOM ViT-L.

Ablation Setting				Combined	On Geom.	Water Column
H	FA	WL	IA			
✓	×	×	×	6.14	<u>1.86</u>	10.30
✓	✓	×	×	1.68	1.90	1.79
✓	✓	✓	×	1.57	<u>1.86</u>	1.54
✓	✓	×	✓	<u>1.53</u>	1.91	<u>1.42</u>
✓	✓	✓	✓	1.51	1.84	1.41

function, as it was the best-performing model from our fine-tuning.

In quantitative evaluations, the most drastic improvement seen in our proposed model is the ability to correctly identify the water column in the depth prediction, as noted in the *Water Column Only* evaluation. Our models perform competitively with or better than existing methods in the on-geometry evaluation. The *Combined* metric, which encapsulates the overall performance of each model, demonstrates that our proposed training improves stereo depth estimation in underwater environments.

Figure 7 shows qualitative comparisons of the small models and Figure 8 shows qualitative comparisons of the large models. The qualitative results demonstrate that existing stereo depth estimation methods fail to transfer to underwater settings, producing either noisy depth estimates or systematic errors such as failure to estimate depth in the water column. Our model is able to remove the water column from challenging images across different water column types and haze levels.

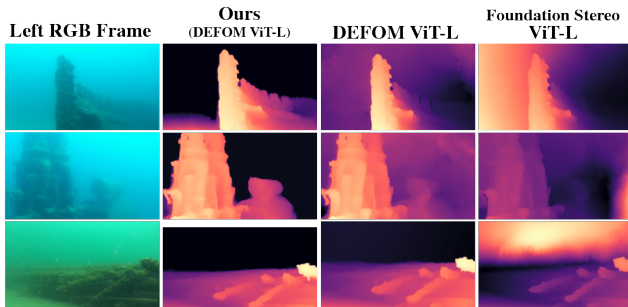


Figure 8. Qualitative results of the Large Vision Transformer (ViT-L) models. Ours, based on DEFOM ViT-L, balances strong on-geometry performance with effective background removal.

TABLE 5

Estimated trajectories are compared against the ground truth. The reported metric is Root Mean Square Absolute Pose Error (APE). Each result is the median of five runs. MAST3R-SLAM is a monocular method, so scale-invariant trajectory evaluations were conducted for MAST3R-SLAM only. Lower is better.

Method	Platform	Engine	Boiler	Long
DROID-SLAM [68]	CPU+GPU	0.164	3.121	5.794
MASt3R SLAM [40]	CPU+GPU	0.395	<u>0.462</u>	1.963
ORB-SLAM3 [8]	CPU	1.662	2.844	6.783
SVIn2 [49]	CPU	1.085	2.413	2.774
TURTLMap [66]	CPU	0.399	0.528	<u>1.312</u>
Ours	CPU+GPU	<u>0.216</u>	0.246	0.444

7.1. Ablation Study on Stereo Depth Estimation

We perform an ablation over the key components of the proposed stereo network fine-tuning. The evaluation of the ablation is done by comparing the EPE on the *On Geometry*, *Water Column*, and *Combined* metrics. The ablation results are shown in Table 4.

Each ablation removes a single component while keeping all others fixed, allowing isolation of its effect on geometry estimation and water-column estimation. The ablation reveals three distinct effects. Full augmentations (FA) are necessary to reduce predictions in the water column. The self-supervised warping loss (WL) primarily improves depth accuracy on observed geometry, while the inclusion of in-air data (IA) prevents degradation under heavy augmentation. The combination of WL and IA yields the best trade-off between geometric fidelity and incorrect predictions on the water column.

8. SLAM Evaluation Results

We evaluate the trajectories from each SLAM method. ORB-SLAM3 is excluded from mapping metrics since the sparse keypoint map is not intended to be used as a structural map. We note that VGGT-SLAM, which is a calibration-free monocular SLAM method [37], was unable to produce a trajectory estimate on any of the sequences and is thus excluded from the result tables. For SVIn2, the mapping metrics are obtained using the dense fusion method proposed in the authors’ follow-up work [76].

8.1. Trajectory Tracking Evaluation

Results of the SLAM trajectory evaluation are shown in Table 5. Our proposed method shows the best performance on the *Boiler* and *Long* SLAM sequences, and second best on the *Engine* sequence. TURTLMap, which does not apply global registration, produces reasonable trajectories on all sequences. Yet, without a mechanism to mitigate accumulated drift, it underperforms against meth-

TABLE 6

Ablation results for disabling individual sensors from the SLAM pipeline. Disabling the camera corresponds to running TURTLMaP [66]. The reported metric is RMS APE; each result is the median of five runs.

Camera	DVL	Baro.		Engine	Boiler	Long
✓	✓	×		<u>0.226</u>	0.246	0.386
✓	×	✓		14.439	102.070	450.411
×	✓	✓		0.399	0.528	1.312
✓	✓	✓		0.216	0.246	<u>0.444</u>

ods that are able to successfully register against a global map. Of the purely vision-based methods, DROID-SLAM and MAST3R-SLAM perform the best. This indicates that even without fine-tuning, learning-based SLAM is better equipped to deal with the challenges of the underwater domain compared to ORB-SLAM3, which uses carefully tuned feature-matching heuristics. SVIn2 converges on all sequences but drifts significantly. We hypothesize that this drift results from the IMU initialization procedure in SVIn2, which benefits from the robot starting at rest. In our field trials, it was not feasible for the robot to be stationary due to dynamic environmental conditions.

We perform an ablation study of the sensors used for tracking, shown in Table 6. The results demonstrate that the DVL plays a critical role in tracking, as disabling it leads to substantial drift across all sequences. Disabling the camera corresponds to running TURTLMaP [66] and degrades performance, particularly on the *Long* sequence, demonstrating the benefit of global registration for reducing accumulated drift. In contrast, the barometer has a relatively small effect on tracking performance: disabling it produces similar results on *Engine* and *Boiler* and slightly improves performance on *Long*. We hypothesize that this is due to noise or bias in the barometer measurements, which can occasionally introduce inconsistent depth constraints when the remaining sensors already provide sufficient information to estimate the trajectory.

8.2. Mapping Evaluation

Quantitative map evaluations are presented in Table 7. We present the mapping metrics from the trajectories evaluated on the desktop platform, and the results indicate that SurfSLAM has comparable or better performance across these metrics. On the completion and recall metrics, SurfSLAM demonstrates significant improvements, indicating more complete reconstructions. For the accuracy and precision metrics, performance is degraded on the *Engine* and *Long* sequences due to a few erroneous pose estimates, resulting in a misaligned projected depth map. This is supported by the qualitative results shown in Figure 9, where the map

TABLE 7

Maps were evaluated using photogrammetry ground-truth. The TURTLMaP column indicates accumulating depths from the TURTLMaP trajectory estimates using a baseline DEFOM ViT-S stereo depth estimator. In contrast, TURTLMaP Ours indicates using the TURTLMaP trajectories and our proposed DEFOM ViT-S stereo depth estimator. All numbers are the median of five runs. For accuracy and completeness, lower is better. For precision and recall, higher is better.

		DROID-SLAM	MASt3R-SLAM	SVIn2	TURTLMaP Base	TURTLMaP Ours	Ours
Boiler	Acc.	0.29	0.44	1.75	1.05	0.48	<u>0.30</u>
	Comp.	0.39	<u>0.09</u>	0.16	0.16	0.15	0.04
	Prec.	0.17	<u>0.32</u>	0.14	0.25	0.26	0.39
	Rec.	0.07	<u>0.82</u>	0.63	0.42	0.41	0.96
Engine	Acc.	0.04	0.53	0.67	<u>0.09</u>	0.14	0.34
	Comp.	0.35	0.20	<u>0.17</u>	0.28	0.60	0.05
	Prec.	0.98	0.29	0.13	<u>0.74</u>	0.55	0.52
	Rec.	0.14	<u>0.79</u>	0.66	0.46	0.32	0.98
Long	Acc.	1.63	0.82	1.86	0.19	0.19	0.50
	Comp.	2.08	5.50	<u>0.18</u>	0.25	0.25	0.03
	Prec.	0.01	0.15	0.09	0.58	<u>0.48</u>	0.47
	Rec.	0.00	0.26	<u>0.62</u>	0.39	0.38	0.99

produced by SurfSLAM shows consistently more accurate measurements and fewer measurements misaligned with the larger reconstructed map. On the *Long* sequence, which features the most low-texture regions, the advantages of SurfSLAM are most apparent, producing the most complete reconstruction. DROID-SLAM excels in accuracy and precision on the feature-rich *Engine* scene. Yet, the completeness and recall are comparatively worse than those of SurfSLAM, indicating less complete maps. SVIn2’s dense fusion struggles, likely due to its difficulty with trajectory estimation (See Table 5). Running TURTLMaP with our stereo network is comparable to SurfSLAM, with a consistent split across metrics. We show consistent improvement in the completeness of the reconstructions with comparable accuracy on longer sequences. Global registration trades a small amount of local accuracy for substantially more complete reconstructions.

Qualitative mapping results are shown in Figure 9. Our method produces the most complete and geometrically consistent reconstruction among the compared approaches. For DroidSLAM, the trajectory estimate diverges as the robot traverses the boiler and loses tracking, leading to an incomplete map, consistent with the quantitative results. For MAST3R-SLAM, despite achieving the second-best trajectory accuracy, depth prediction errors introduce noticeable artifacts and reduce map accuracy. For SVIn2 and TURTLMaP, accumulated drift distorts and misaligns the reconstruction: SVIn2 exhibits pronounced distortion on

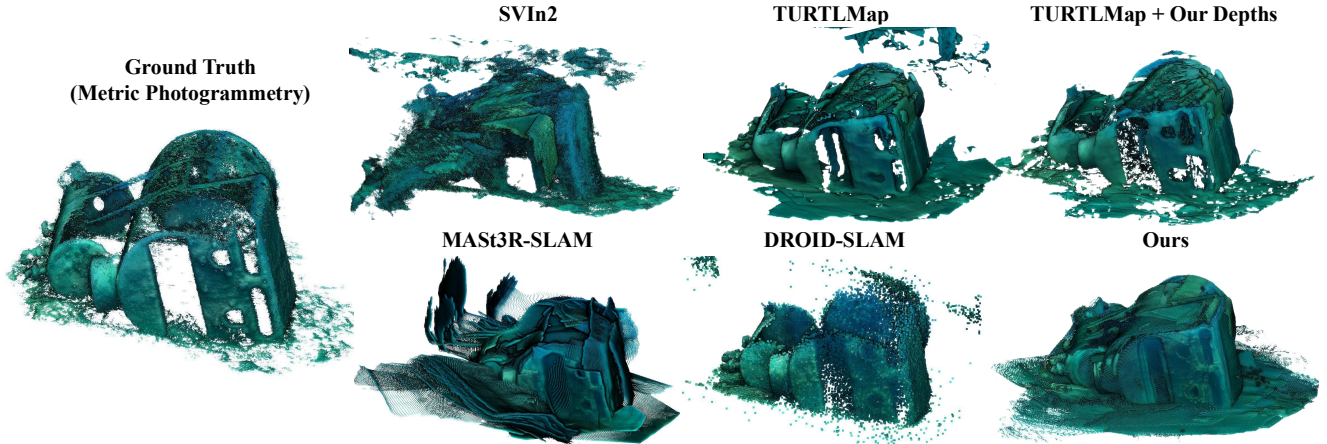


Figure 9. Qualitative comparison of the maps generated by each method on the *Boiler* sequence. Methods that produce vastly incorrect maps due to their incorrect pose estimates (specifically, DroidSLAM [68], SVIn2 [49, 76], and TUTLMMap [66]) are cropped for visualization purposes. The SVIn2 image refers to running the SVIn2 [49] for trajectory estimation and using the dense mapping method from [76]. Our map produces a more accurate, cleaner, and more complete map than those produced by the baseline methods.

TABLE 8

We compare the tracking performance of our method on each of the three sequences on two platforms, a high-performance desktop machine and an NVIDIA Jetson Thor. The reported metric is Root Mean Square Absolute Pose Error (APE).

Platform	Engine	Boiler	Long
Desktop	0.216	0.246	0.444
Jetson Thor	0.171	0.216	0.333

the left side of the map, while TUTLMMap shows missing geometry due to depth misalignment, causing valid observations to be filtered during mapping. Finally, the map produced by TUTLMMap using our proposed depths is comparable to the maps produced by TUTLMMap alone.

8.3. Compute Comparison and Runtime

This section compares the performance of our method between the desktop platform and the NVIDIA Jetson Thor.

Table 8 shows that SurfSLAM achieves comparable performance on both desktop and embedded platforms, reinforcing its suitability for field deployment. On all three sequences, the Jetson yields slightly lower tracking error than the desktop. Because the Jetson drops more frames during stereo matching (see Table 9), a plausible explanation is that it skips a frame that would otherwise induce an erroneous registration on the desktop platform.

We present the runtime of key components of the algorithm in Table 9. On each of the platforms used to evaluate the system, we report two key timing metrics: time for stereo inference and time to solve the backend optimization. In practice, several of the components run in parallel.

To ensure real-time operation, each component drops stale data. Table 9 also reports statistics on how many frames are dropped within the system. On the desktop platform, 98.4% of input stereo frames are processed by the stereo inference module. All of those frames are processed by the registration module to search for global registrations. On the Jetson platform, the same 98.4% of the input stereo frames are processed. However, due to the slower stereo inference time, 22.8% of frames processed by the stereo matcher become stale and are dropped before registration is attempted.

9. Discussion & Conclusion

This paper presents a comprehensive methodology for stereo-based state estimation and scene reconstruction in challenging underwater environments. We first introduced a sim-to-real training pipeline for underwater stereo disparity estimation and demonstrated that it outperforms the state-of-the-art in underwater domains. Next, we presented SurfSLAM, a method that fuses stereo images with IMU, DVL, and barometer measurements to perform long-term state estimation and mapping in real-world underwater environments. In addition to the algorithm contributions of this paper, we present two new datasets for underwater stereo perception and underwater SLAM. The first dataset comprises both a curated dataset of scenes that mimic a variety of underwater environments and an augmentation pipeline that randomizes underwater imaging effects. We also present a comprehensive dataset of real-world shipwrecks.

The results of our experiments demonstrate state-of-the-art performance on stereo estimation in the underwater domain. While several related works simulate underwater haze, we additionally simulate directional lighting, caustics,

TABLE 9

We report the runtime of both stereo inference and the backend optimization (mean $\pm$ standard deviation), averaged across all sequences and 5 trials. During real-time operation of the system, stale frames are dropped to prevent the system from falling behind. For instance, an N% registration drop rate means that of all the frames the registration module received, it discarded N% of the frames and processed the rest.

Group	Metric	Ours (Desktop)	Ours (Jetson)
Timing	Stereo Inference	359.3 $\pm$ 26.4 ms	786.1 $\pm$ 51.2 ms
	Optimization	6.7 $\pm$ 3.4 ms	9.9 $\pm$ 5.1 ms
Drops	Stereo	1.6 %	1.6 %
	Registration	0.0 %	21.5 %
	Overall	1.6 %	22.8 %

and suspended particles. Experiments demonstrate that this comprehensive simulation improves performance over the haze-only simulation. Further, while baseline methods most often train either on simulated data or via self-supervised training on real underwater images, we demonstrate that a combined approach yields superior results.

The results of our SLAM evaluation support the approach of relying on acoustic and inertial sensors as the primary local tracker, while using a highly accurate but slow vision model to correct drift when possible. The trajectory evaluation results support the conclusion that this leads to stronger performance in challenging environments than methods that rely on feature tracking, which is susceptible to failure in low-texture underwater scenes.

A limitation of our existing evaluations arises from the difficulty of reconstructing metric ground truth from images under highly turbid conditions. While we hypothesize that our sim-to-real pipeline will have a greater impact in more turbid water, it is challenging to test this hypothesis because photogrammetry methods cannot reconstruct ground truth in such conditions. A potential avenue for future work is to use other sensors, such as 3D SONAR or Underwater LiDAR, to generate high-quality ground truth in such environments.

An avenue for future work to improve SurfSLAM is to augment the SLAM pipeline with bundle adjustment rather than relying on frame-to-frame registration. In practice, this is challenging in our target domain, which includes large, textureless regions that can push the stability of existing bundle-adjustment methods.

References

- [1] OpenCV: Camera Calibration and 3D Reconstruction. 5
- [2] Agisoft LLC. Agisoft Metashape Professional, Version 1.4.5, 2018. Software. 3
- [3] Derya Akkaynak and Tali Treibitz. A revised underwater image formation model. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6723–6732, 2018. 2
- [4] Loïca Avanthey and Laurent Beaudoin. Dense In Situ Underwater 3D Reconstruction by Aggregation of Successive Partial Local Clouds. *Remote Sensing*, 16(24):4737, 2024. 3
- [5] Ayoung Kim and R. Eustice. Pose-graph visual SLAM with geometric model selection for autonomous underwater ship hull inspection. In *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1559–1565, St. Louis, MO, 2009. IEEE. 3
- [6] Dana Berman, Deborah Levy, Shai Avidan, and Tali Treibitz. Underwater single image color restoration using haze-lines and a new quantitative dataset. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(8):2822–2837, 2020. 3, 6
- [7] Gideon Billings, Richard Camilli, and Matthew Johnson-Roberson. Hybrid visual slam for underwater vehicle manipulator systems. *IEEE Robotics and Automation Letters*, 7(3):6798–6805, 2022. 6, 13
- [8] Carlos Campos, Richard Elvira, Juan J. Gómez Rodríguez, José M. M. Montiel, and Juan D. Tardós. Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021. 2, 13, 15
- [9] Luca Carlone, Zsolt Kira, Chris Beall, Vadim Indelman, and Frank Dellaert. Eliminating conditionally independent sets in factor graphs: A unifying perspective based on smart factors. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4290–4297, 2014. 12
- [10] Xiangning Chen, Cho-Jui Hsieh, and Boqing Gong. When vision transformers outperform resnets without pretraining or strong data augmentations. In *International Conference on Learning Representations*, 2022. 13
- [11] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superpoint: Self-supervised interest point detection and description. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 337–33712, 2018. 9
- [12] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021. 13
- [13] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE Transactions on Big Data*, pages 1–17, 2025. 10
- [14] Epic Games. Fab, 2024. Epic’s unified marketplace for digital assets. [Online]. Available: <https://www.fab.com/>. 5
- [15] Ryan Eustice, Hanumant Singh, John Leonard, Matthew Walter, and Robert Ballard. Visually Navigating the RMS Titanic with SLAM Information Filters. In *Robotics: Science and Systems I*. Robotics: Science and Systems Foundation, 2005. 3
- [16] Christian Forster, Luca Carlone, Frank Dellaert, and Davide Scaramuzza. On-manifold preintegration for real-time

- visual-inertial odometry. *IEEE Transactions on Robotics*, 33(1):1–21, 2017. 5, 9
- [17] Paul Furgale. Representing robot pose: The good, the bad, and the ugly, 2025. [Online]. Available: <https://rpg.ifi.uzh.ch/docs/teaching/2025/FurgaleTutorial.pdf>. 5
- [18] Ross Girshick. Fast r-cnn, 2015. 8
- [19] Clément Godard, Oisín Mac Aodha, and Gabriel J. Brostow. Unsupervised monocular depth estimation with left-right consistency. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6602–6611, 2017. 8
- [20] Michael Grupp. evo: Python package for the evaluation of odometry and slam. <https://github.com/MichaelGrupp/evo>, 2017. 13
- [21] Heiko Hirschmüller. Stereo processing by semiglobal matching and mutual information. *IEEE Transactions on pattern analysis and machine intelligence*, 30(2):328–341, 2007. 2
- [22] Heiko Hirschmüller. Stereo processing by semiglobal matching and mutual information. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(2):328–341, 2008. 3
- [23] Shitong Hou, Dai Jiao, Bin Dong, Haochen Wang, and Gang Wu. Underwater inspection of bridge substructures using sonar and deep convolutional network. *Advanced Engineering Informatics*, 52:101545, 2022. 2
- [24] Peter J Huber. Robust estimation of a location parameter. In *Breakthroughs in statistics: Methodology and distribution*, pages 492–518. Springer, 1992. 10
- [25] Seth Isaacson, Pou-Chun Kung, Mani Ramanagopal, Ram Vasudevan, and Katherine A. Skinner. Loner: Lidar only neural representations for real-time slam. *IEEE Robotics and Automation Letters*, 8(12):8042–8049, 2023. 13
- [26] Krishna Murthy Jatavallabhula, Edward Smith, Jean-François Lafleche, Clement Fuji Tsang, Artem Rozantsev, Wenzheng Chen, Tommy Xiang, Rev Lebaredian, and Sanja Fidler. Kaolin: A pytorch library for accelerating 3d deep learning research, 2019. 6
- [27] Nils Gunnar Jerlov. *Optical oceanography*. Elsevier, 1968. 2, 6
- [28] Hualie Jiang, Zhiqiang Lou, Laiyan Ding, Rui Xu, Minglang Tan, Wenjie Jiang, and Rui Huang. Defom-stereo: Depth foundation model based stereo matching. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21857–21867, 2025. 2, 3, 7, 13, 14
- [29] Hervé Jégou, Matthijs Douze, Cordelia Schmid, and Patrick Pérez. Aggregating local descriptors into a compact image representation. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 3304–3311, 2010. 9
- [30] Michael Kaess, Hordur Johannsson, Richard Roberts, Violella Ila, John J Leonard, and Frank Dellaert. isam2: Incremental smoothing and mapping using the bayes tree. *The International Journal of Robotics Research*, 31(2):216–235, 2012. 9
- [31] Ayoung Kim and Ryan M. Eustice. Combined visually and geometrically informative link hypothesis for pose-graph visual SLAM using bag-of-words. In *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1647–1654, San Francisco, CA, 2011. IEEE. 3
- [32] Ayoung Kim and Ryan M. Eustice. Real-Time Visual SLAM for Autonomous Underwater Hull Inspection Using Visual Saliency. *IEEE Trans. Robot.*, 29(3):719–733, 2013. 3
- [33] Vincent Leroy, Yohann Cabon, and Jerome Revaud. Grounding image matching in 3d with mast3r. In *Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part LXXII*, page 71–91, Berlin, Heidelberg, 2024. Springer-Verlag. 2
- [34] Jie Li, Katherine A. Skinner, Ryan M. Eustice, and Matthew Johnson-Roberson. Watergan: Unsupervised generative network to enable real-time color correction of monocular underwater images. *IEEE Robotics and Automation Letters*, page 1–1, 2017. 2
- [35] Lahav Lipson, Zachary Teed, and Jia Deng. RAFT-Stereo: Multilevel recurrent field transforms for stereo matching. In *International Conference on 3D Vision (3DV)*, 2021. 3, 13
- [36] Qingxuan Lv, Junyu Dong, Yuezun Li, Sheng Chen, Hui Yu, Shu Zhang, and Wenhan Wang. UW Stereo: A large synthetic dataset for underwater stereo matching. *IEEE Transactions on Circuits and Systems for Video Technology*, 2025. 4, 6
- [37] Dominic Maggio, Hyungtae Lim, and Luca Carlone. Vggt-slam: Dense rgb slam optimized on the sl (4) manifold. *Advances in Neural Information Processing Systems*, 38:129839–129867, 2026. 3, 13, 15
- [38] Aaron Marburg and Marc Micatka. A dataset for the assessment of underwater slam degradation in turbid water. In *OCEANS 2025 Chicago*, 2025. 6
- [39] Nikolaus Mayer, Eddy Ilg, Philip Häusser, Philipp Fischer, Daniel Cremers, Alexey Dosovitskiy, and Thomas Brox. A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4040–4048, 2016. 12
- [40] Riku Murai, Eric Dexheimer, and Andrew J. Davison. Mast3r-slam: Real-time dense slam with 3d reconstruction priors. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16695–16705, 2025. 3, 13, 15
- [41] NVIDIA Corporation. Omniverse. Software. 5
- [42] NVIDIA Corporation. Nvidia isaac sim (version 4.5.1). GitHub repository, 2025. Available at <https://github.com/isaac-sim/IsaacSim>. 5
- [43] Paul Ozog and Ryan M. Eustice. Real-time SLAM with piecewise-planar surface models and sparse 3D point clouds. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1042–1049, Tokyo, 2013. IEEE. 3
- [44] Shu Pan, Ziyang Hong, Zhangrui Hu, Xiandong Xu, Wenjie Lu, and Liang Hu. Russo: Robust underwater slam with sonar optimization against visual degradation. *IEEE/ASME Transactions on Mechatronics*, 30(6):5456–5467, 2025. 3
- [45] Charilaos Papadopoulos and Georgios Papaioannou. Realistic real-time underwater caustics and godrays. *19th International Conference on Computer Graphics and Vision, GraphiCon'2009 - Conference Proceedings*, 2010. 7

- [46] S.M. Pizer, R.E. Johnston, J.P. Ericksen, B.C. Yankaskas, and K.E. Muller. Contrast-limited adaptive histogram equalization: speed and effectiveness. In *[1990] Proceedings of the First Conference on Visualization in Biomedical Computing*, pages 337–345, 1990. 9
- [47] E. Potokar, S. Ashford, M. Kaess, and J. Mangelson. HoloOcean: An underwater robotics simulator. In *Proc. IEEE Intl. Conf. on Robotics and Automation, ICRA*, Philadelphia, PA, USA, 2022. 5
- [48] Sharmin Rahman, Alberto Quattrini Li, and Ioannis Rekleitis. Contour based reconstruction of underwater structures using sonar, visual, inertial, and depth sensor. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8054–8059, 2019. 3
- [49] Sharmin Rahman, Alberto Quattrini Li, and Ioannis Rekleitis. Svin2: A multi-sensor fusion-based underwater slam system. *The International Journal of Robotics Research*, 41 (11-12):1022–1042, 2022. 3, 6, 13, 15, 17
- [50] Yelena Randall, Ori Lifschitz, and Tali Treibitz. Flsea: Underwater visual-inertial and stereovision forward-looking data sets. *Journal of Field Robotics*, n/a(n/a), 2026. 3, 6
- [51] Joern Rehder, Janosch Nikolic, Thomas Schneider, Timo Hinzmann, and Roland Siegwart. Extending kalibr: Calibrating the extrinsics of multiple imus and of individual axes. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4304–4311, 2016. 12
- [52] Edgar Riba, Dmytro Mishkin, Daniel Ponsa, Ethan Rublee, and Gary Bradski. Kornia: an open source differentiable computer vision library for pytorch. In *Winter Conference on Applications of Computer Vision*, 2020. 8
- [53] Blake Romrell, Abigail Austin, Braden Meyers, Ryan Anderson, Carter Noh, and Joshua G Mangelson. A preview of holoocean 2.0. *arXiv preprint arXiv:2510.06160*, 2025. 5
- [54] Monika Roznere, Philippos Mordohai, Ioannis Rekleitis, and Alberto Quattrini Li. 3-d reconstruction using monocular camera and lights: Multi-view photometric stereo for non-stationary robots. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1026–1032, 2023. 6
- [55] Paul-Edouard Sarlin, Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superglue: Learning feature matching with graph neural networks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4937–4946, 2020. 9, 10
- [56] Daniel Scharstein and Richard Szeliski. A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. *International journal of computer vision*, 47:7–42, 2002. 3
- [57] Y.Y. Schechner and N. Karpel. Recovery of underwater visibility and structure by polarization analysis. *IEEE Journal of Oceanic Engineering*, 30(3):570–587, 2005. 5
- [58] Johannes L. Schönberger, Enliang Zheng, Jan-Michael Frahm, and Marc Pollefeys. Pixelwise view selection for unstructured multi-view stereo. In *Computer Vision – ECCV 2016*, pages 501–518, Cham, 2016. Springer International Publishing. 3, 12
- [59] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4104–4113, 2016. 3, 12
- [60] Aleksandr Segal, Dirk Hähnel, and Sebastian Thrun. Generalized-icp. In *Robotics: Science and Systems*. The MIT Press, 2009. 10
- [61] Advait Venkatraman Sethuraman, Manikandasri Ram Srinivasan Ramanagopal, and Katherine A Skinner. WaterNeRF: Neural radiance fields for underwater scenes. In *OCEANS 2023-MTS/IEEE US Gulf Coast*, pages 1–7. IEEE, 2023. 5
- [62] Md Abu Bakr Siddique, Jiayi Wu, Ioannis Rekleitis, and Md Jahidul Islam. Aquafuse: Waterbody fusion for physics-guided view synthesis of underwater scenes. *IEEE Robotics and Automation Letters*, 10(5):4316–4323, 2025. 6
- [63] Kurran Singh, Jungseok Hong, Nicholas R Rypkema, and John J Leonard. Opti-acoustic semantic slam with unknown objects in underwater environments. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1169–1176. IEEE, 2024. 2
- [64] Sketchfab, Inc. Sketchfab, 2025. Online platform for publishing, sharing, and discovering 3D content. [Online]. Available: <https://sketchfab.com>. 5
- [65] Katherine A. Skinner, Junming Zhang, Elizabeth A. Olson, and Matthew Johnson-Roberson. UW StereoNet: Unsupervised learning for depth estimation and color correction of underwater stereo imagery. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 7947–7954, 2019. 2, 4, 6, 8
- [66] Jingyu Song, Onur Bagoren, Razan Andigani, Advait Sethuraman, and Katherine A Skinner. Turtlmap: Real-time localization and dense mapping of low-texture underwater environments with a low-cost unmanned underwater vehicle. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1191–1198. IEEE, 2024. 3, 9, 13, 15, 16, 17
- [67] Jingyu Song, Haoyu Ma, Onur Bagoren, Advait V. Sethuraman, Yiting Zhang, and Katherine A. Skinner. OceanSim: A GPU-accelerated underwater robot perception simulation framework. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. 5, 6
- [68] Zachary Teed and Jia Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. *Advances in neural information processing systems*, 34:16558–16569, 2021. 2, 10, 13, 15, 17
- [69] Alexander Thoms, Gabriel Earle, Nicholas Charron, and Sri Ram Narasimhan. Tightly coupled, graph-based dvl/imu fusion and decoupled mapping for slam-centric maritime infrastructure inspection. *IEEE Journal of Oceanic Engineering*, 48(3):663–676, 2023. 3, 9
- [70] C. Upstill. Light caustics from rippling water. *Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences*, 365(1720):95–104, 1979. 6
- [71] Madhu Vankadari, Samuel Hodgson, Sangyun Shin, Kaichen Zhou, Andrew Markham, and Niki Trigoni. Dusk till dawn: Self-supervised nighttime stereo depth estimation using visual foundation models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 17976–17982, 2024. 8

- [72] Elizabeth Vargas, Raluca Scona, Jonatan Scharff Willners, Tomasz Luczynski, Yu Cao, Sen Wang, and Yvan R. Petillot. Robust Underwater Visual SLAM Fusing Acoustic Sensing. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2140–2146, Xi'an, China, 2021. IEEE. 3
- [73] Di Wang and Yong Jie Pang. Research on disparity map generation method of underwater target based on the improved sift algorithm. *Applied Mechanics and Materials*, 741:701–704, 2015. 3
- [74] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5294–5306, 2025. 2
- [75] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. TartanAir: A dataset to push the limits of visual SLAM. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4909–4916, 2020. 12
- [76] Weihan Wang, Bharat Joshi, Nathaniel Burgdorfer, Konstantinos Batsos, Alberto Quattrini Li, Philippos Mordohai, and Ioannis Rekleitis. Real-time dense 3d mapping of underwater environments. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5184–5191. IEEE, 2023. 3, 13, 15, 17
- [77] Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. Foundationstereo: Zero-shot stereo matching. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5249–5260, 2025. 2, 3, 5, 7, 8, 13, 14
- [78] Zhengri Wu, Yiran Wang, Yu Wen, Zeyu Zhang, Biao Wu, and Hao Tang. Stereoadapter: Adapting stereo depth estimation to underwater scenes, 2025. 2, 4, 6
- [79] Gangwei Xu, Xianqi Wang, Zhaoxing Zhang, Junda Cheng, Chunyuan Liao, and Xin Yang. Igev++: Iterative multi-range geometry encoding volumes for stereo matching. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. 2, 3, 13, 14
- [80] Shida Xu, Tomasz Luczynski, Jonatan Scharff Willners, Ziyang Hong, Kaicheng Zhang, Yvan R. Petillot, and Sen Wang. Underwater Visual Acoustic SLAM with Extrinsic Calibration. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7647–7652, Prague, Czech Republic, 2021. IEEE. 3
- [81] Shida Xu, Jonatan Scharff Willners, Joshua Roe, Sean Katagiri, Tomasz Luczynski, Yvan Petillot, and Sen Wang. Tank dataset: An underwater multi-sensor dataset for slam evaluation. *The International Journal of Robotics Research*, page 02783649251364904, 2025. 6
- [82] Shida Xu, Kaicheng Zhang, and Sen Wang. Aqua-slam: Tightly coupled underwater acoustic-visual-inertial slam with sensor calibration. *IEEE Transactions on Robotics*, 41: 2785–2803, 2025. 3
- [83] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. In *Advances in Neural Information Processing Systems*, pages 21875–21911. Curran Associates, Inc., 2024. 3
- [84] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *Advances in Neural Information Processing Systems*, 37:21875–21911, 2024. 2
- [85] Xinchun Ye, Jinyi Zhang, Yazhi Yuan, Rui Xu, Zhihui Wang, and Haojie Li. Underwater depth estimation via stereo adaptation networks. *IEEE Transactions on Circuits and Systems for Video Technology*, 33(9):5089–5101, 2023. 2, 4, 13, 14
- [86] Xinchun Ye, Yue Chang, Rui Xu, and Haojie Li. Uw-adapter: Adapting monocular depth estimation model in underwater scenes. *IEEE Transactions on Multimedia*, 2025. 2
- [87] Boxiao Yu, Jiayi Wu, and Md Jahidul Islam. Udepth: Fast monocular depth estimation for visually-guided underwater robots. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3116–3123, 2023.
- [88] Hongjie Zhang, Gideon Billings, and Stefan B Williams. SPADE: Sparsity adaptive depth estimator for zero-shot, real-time, monocular depth estimation in underwater environments. *arXiv preprint arXiv:2510.25463*, 2025. 2
- [89] Lvwei Zhu, Yakun Ju, and Ying Gao. A novel stereo matching network for underwater scenes. In *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2025. 4